# Cursul 2 - Automate finite deterministe (DFA)

**Definiție**: Un DFA este un tuplu $A = (K, \Sigma, \delta, q_0, F)$, unde:

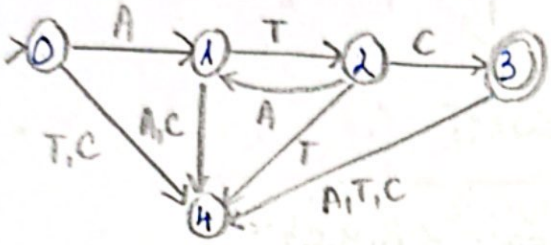
$K$ = mulțimea stărilor

$\Sigma$ = alfabetul de input

$\delta : K \times \Sigma \to K$ = funcția de tranziție

$q_0$ = starea inițială

$F$ = mulțimea stărilor finale, $F \subseteq K$




- pentru inputul ATATC → se ajunge în starea finală, ATATC este un cuvânt acceptat de DFA
- ATATAC → NU e acceptat
- ATATATC → E acceptat
- ATACC → NU e acceptat

Mulțimea tuturor cuvintelor acceptate de un DFA se numește limbajul DFA-ului.

**Definiție**: $\Sigma^*$ este mulțimea tuturor cuvintelor formate peste alfabetul $\Sigma$

**Definiție**: O configurație a unui DFA este o pereche $(q, w) \in K \times \Sigma^*$

**Definiție**: O relație $\vdash_A \subseteq (K, \Sigma^*) \times (K, \Sigma^*)$ se numește one-step relation și se definește:

$$(q, cw) \vdash_A (q', w) \text{ if } \delta(c, q) = q'$$

ex: (pt automatul de mai sus)

$(1, TATTC) \vdash_A (2, ATTC) \vdash_A (1, TTC) \vdash_A (2, TC) \vdash_A (4, C) \vdash_A (4, \varepsilon)$

**Definiție**: Relația $\vdash_A^* \subseteq (K, \Sigma^*) \times (K, \Sigma^*)$ se numește zero-or-more steps relation.

ex: (de mai sus) $(1, TATTC) \vdash_A^* (4, \varepsilon)$



Scanned with CamScanner

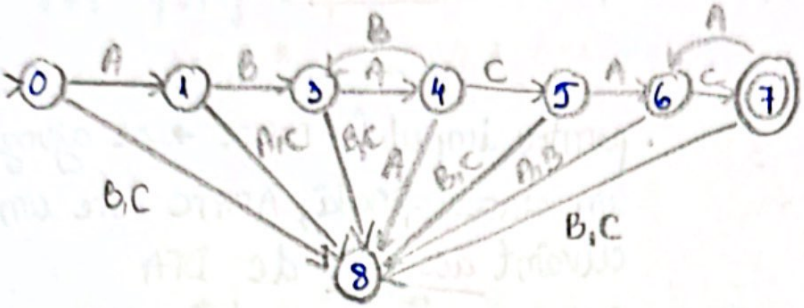
**Definiție:** Un cuvânt w este acceptat de automatul A dacă $(q_0, w) \vdash_A^* (q_f, \varepsilon)$ și $q_f \in F$

**Definiție:** Limbajul L(A) acceptat de A este

$L(A) = \{ w \in \Sigma^* \mid w \text{ e acceptat de } A \}$

Quiz

Fie automatul:




1) Care este alfabetul acestui DFA: $\{A, B, C\}$
2) Funcția delta este totală? NU, nu avem tranziții din starea 8.
3) Configurația accesibilă din (4, BACABA) într-un singur pas este: (3, ACABA)
4) Configurația accesibilă din (4, BACABA) în 5 pași este: (8, A)
5) Cuvântul ABACAB este acceptat de DFA?: NU

## Cursul 3 - Expresii regulate

**Definiție:** Un limbaj peste un alfabet $\Sigma$ este orice submulțime a lui $\Sigma^*$.

DFA-urile sunt o reprezentare finită pentru a descrie unele limbaje.



Scanned with CamScanner

## Operatori

- Kleene Star. Dacă A este o mulțime, atunci

$$A^* = \{\varepsilon\} \cup A \cup A^2 \cup A^3 \dots$$

unde:

$A^i = \{w_1 w_2 \dots w_k \; ; \; w_1, w_2, \dots, w_k \in A\}$

↳ cuvinte de lungimea $i$ cu caract. din $A$.

ex: $\{0,1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$

$\{00,1\}^* = \{\varepsilon, 00, 1, 001, 100, 0000, 11, \dots\}$

- Concatenarea ($\cdot$). Dacă A și B sunt mulțimi, atunci:

$$AB = \{w_1 w_2 \; ; \; w_1 \in A, w_2 \in B\}$$

ex: $A = \{0, 01, 1\}$ ; $B = \{11, 00\}$ ⇒

⇒ $AB = \{011, 000, 0111, 0100, 111, 100\}$

$A = \{0,1\}$ ; $B = \{0\}^*$ ⇒

⇒ $AB = \{0, 00, 000, \dots, 1, 10, 100, \dots\}$

Definiție: Fie $\Sigma$ un alfabet. O expresie regulată peste $\Sigma$ este definită astfel:

- $\emptyset$ este o expresie regulată;
- $\varepsilon$ este o expresie regulată;
- $c$, unde $c \in \Sigma$, este o expresie regulată;
- dacă $e$ este o expresie regulată ⇒ $e^*$ este o expresie regulată;
- dacă $e_1$ și $e_2$ sunt expresii regulate ⇒ $e_1 e_2$ este o expresie regulată;
- dacă $e_1$ și $e_2$ sunt expresii regulate ⇒ $e_1 \cup e_2$ este o expresie regulată.



Scanned with CamScanner

**Definiție**: Semantica unui ~~limbaj~~ expresii regulate este o funcție care atribuie fiecărei expresii regulate un limbaj.

$L(\emptyset) = \emptyset$ ; $L(\varepsilon) = \{\varepsilon\}$ ; $L(c) = \{c\}$

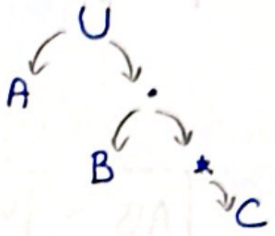
$L(e^*) = L(e)^*$ ; $L(e_1 e_2) = L(e_1)L(e_2)$ ; $L(e_1 \cup e_2) = L(e_1) \cup L(e_2)$

ex: $L((01)^*) = \{\varepsilon, 01, 0101, \dots\}$ $(= L(01)^* = (L(0)L(1))^* = (\{0\}\cdot\{1\})^* = \{0,1\}^*)$

$L((0 \cup 11)^*) = \{\varepsilon, 0, 11, 00, 011, 110, 1111, \dots\}$

Precedența cu care efectuăm operațiile este:

1. Kleene star
2. Concatenation
3. Union




ex: $L(A \cup BC^*) = L(A) \cup L(BC^*) =$

$= \{A\} \cup L(B) \cdot L(C^*) =$

$= \{A\} \cup \{B\} \cdot \{\varepsilon, C, CC, \dots\} =$

$= \{A, B, BCC, BC, \dots\}$

ex: Scrieți un regex pt:

$L_1 = \{w \in \{A,T,C\}^* ; w = (AT)^n C, \text{ unde } n > 0\}$

$AT(AT)^*C = AT^+C$

$ee^* = e^+$

$e \cup \varepsilon = e?$

ex: Scrieți un regex pt:

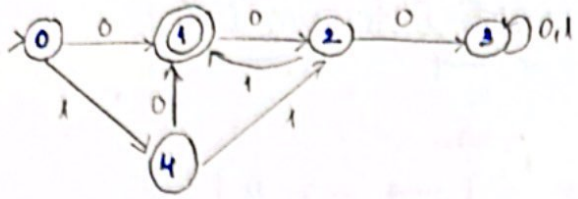
$L_2 = \{w \in \{0,1\}^* ;$ w începe și se termină ori cu o secvență de 0, ori cu o secvență de 1$\}$

$(0^+ \cup 1^+)(0 \cup 1)^* (0^+ \cup 1^+)$



Scanned with CamScanner

ex: Scrieți un regex pt limbajul acceptat de automatul:




$(0 \cup 10 \cup 111)(01)^*$

Quiz

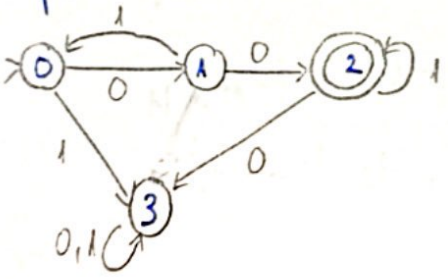
1. Selectați din lista de mai jos expresiile regulate care generează același limbaj

- ☑ $(0^*1^*)^*$
- ☐ $0^*1^*$
- ☑ $(0 \cup 1)^*$ → generează orice cuvânt liniar
- ☐ $1^*0^*$
- ☑ $(1^*0^*)^*$

2. Scrieți o expresie regulată pentru limbajul tuturor cuvintelor liniare care încep și se termină cu același bit.

$0(0 \cup 1)^*0 \cup 1(0 \cup 1)^*1 \cup 0 \cup 1$

3. Scrieți o expresie regulată care generează limbajul acceptat de automatul:



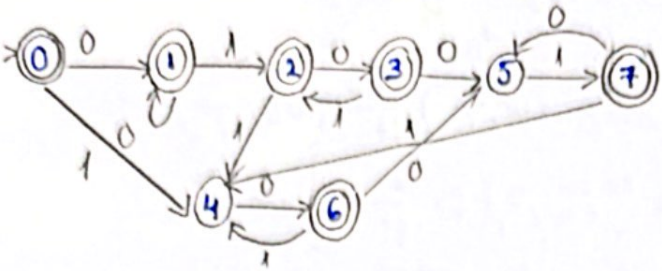

$(01)^*00\,1^*$ sau $0(10)^*0\,1^*$.



Scanned with CamScanner

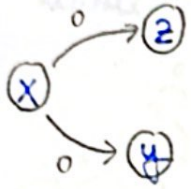
# Cursul 4 - Transformarea Regex - NFA.

Conversia regexului $0^*(01 \cup 10)^*$ într-un DFA:

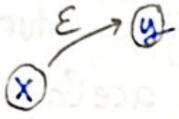



Conversia Regex - DFA nu e straight forward la cazul general.
Se introduce un nou tip de automat, automatul finit nondeterministic (NFA), cu care conversia se face mai ușor.

La NFA avem:

- 

$(x, 0w) \vdash_N (z, w)$
și în același timp
$(x, 0w) \vdash_N (y, w)$
→ un NFA poate tranziționa în mai multe stări pe același input

- 

$(x, w) \vdash_N (y, w)$ → la un pas, un NFA poate să-și schimbe starea fără să consume din input.

**Definiție**: Un NFA este un tuplu:

$$N = (K, \Sigma, \Delta, q_0, F)$$

unde:

$K$ = mulțimea stărilor
$\Sigma$ = alfabetul de input
$\Delta \subseteq (K \times \Sigma^* \times K)$ = relația de tranziție
$q_0$ = starea inițială
$F$ = mulțimea stărilor finale, $F \subseteq K$

ex: (de mai sus): $(x, 0, y) \in \Delta$; $(x, 0, z) \in \Delta$



Scanned with CamScanner

**Definiție:** O configurație a unui NFA este o pereche $(q, w) \in K \times \Sigma^*$

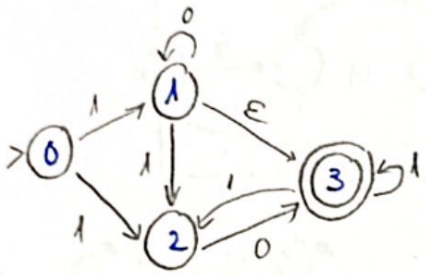
**Definiție:** Relația $\vdash_N \subseteq (K, \Sigma^*) \times (K, \Sigma^*)$ se numește one-step relation și e definită:
$(q, cw) \vdash_N (q', w)$ dacă $(q, c, q') \in \Delta$

**Definiție:** Relația $\vdash_N^* \subseteq (K, \Sigma^*) \times (K, \Sigma^*)$ se numește zero-or-more steps relation

**Definiție:** Un cuvânt w este acceptat de N dacă $(q_0, w) \vdash_N^* (q_f, \varepsilon)$, unde $q_f \in F$.

**Definiție:** Limbajul L(N) acceptat de N este:
$\{w \in \Sigma; w \text{ este acceptat de } N\}$

ex:




- din starea 0 se poate merge în stările 1 și 2 pe input 1
- ——"—— 3 ——"——
  ——"—— 2 și 3 ——"——
- nu avem tranziție pe 0 din starea 3
- tranziția pe ε din starea 1

$(0, 10101) \vdash_N (1, 0101) \vdash_N (1, 101) \vdash_N (2, 01)$
sau $\vdash_N (3, 1101) \vdash_N (3, 01)$ stuck
sau $\vdash_N (3, 010101) \to$ STUCK
sau $\vdash_N (2, 0101) \vdash_N (3, 101)$ ...

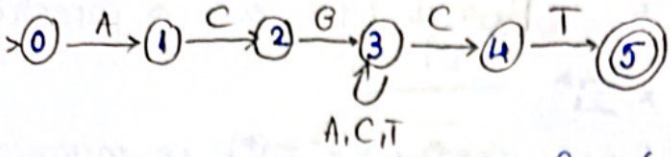
sau $\vdash_N (2, 010101) \vdash_N (3, 1101) \vdash_N (3, 01)$ stuck
sau $\vdash_N (2, 0101) \vdash_N (3, 101) \vdash_N$

$\vdash_N (3, 01)$ stuck
sau $\vdash_N (2, 01) \vdash_N (3, 1) \vdash_N (3, \varepsilon)$ cuv acceptat
sau $\vdash_N (2, \varepsilon)$ cuv nu e acceptat

$\Rightarrow 10101 \in L(N)$

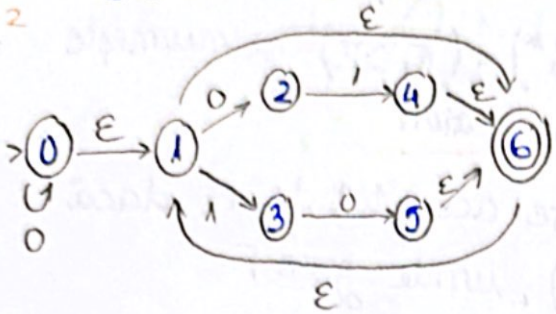


Scanned with CamScanner

ex: 


Automatul codifică limbajul $L(ACG(AUCUT)^*CT)$.

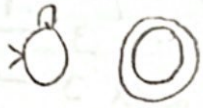
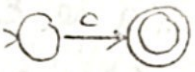
ex: Să se convertească regexul $0^*(01 \cup 10)^*$ într-un NFA.



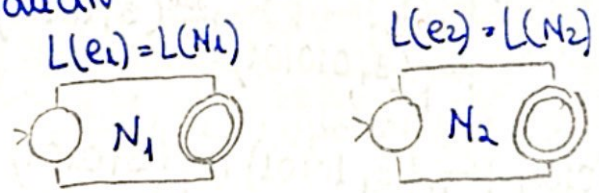

## Algoritm pt. transformarea Regex-NFA

Pentru un regex dat e, construim un NFA N cu o stare finală a.î. $L(e) = L(N)$

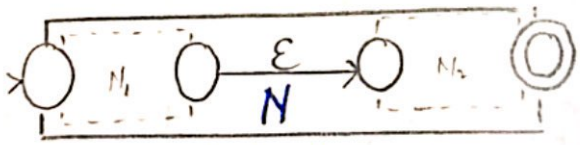
I. caz de bază:

- $\emptyset$ 
- $\varepsilon$ 
- $c \in \Sigma$ 


II pasul inductiv

- $e_1 e_2$ : 


devine

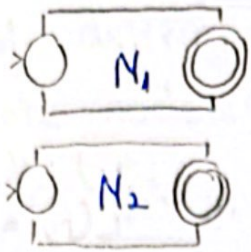



$L(N) = L(e_1 e_2)$



Scanned with CamScanner

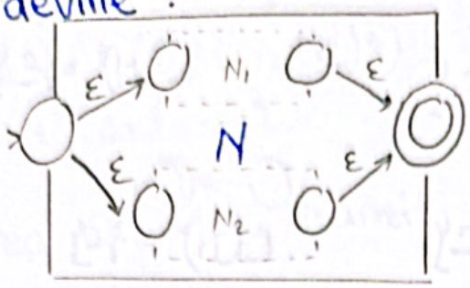
- $e_1 \cup e_2$:




$L(e_1) = L(N_1)$

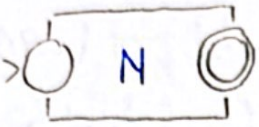
$L(e_2) = L(N_2)$

devine:



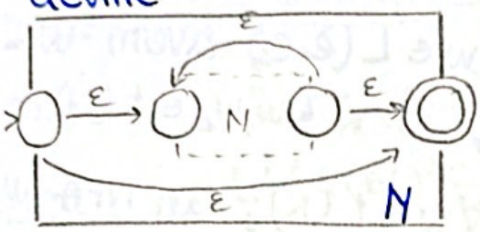

$L(N) = L(e_1 \cup e_2)$

- $e^*$:



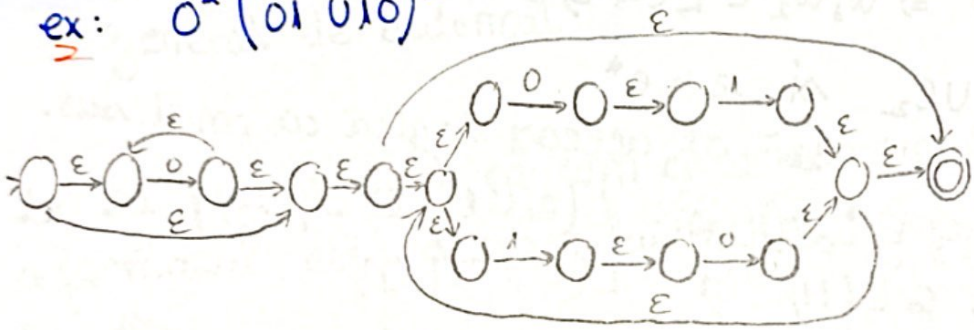

$L(N) = L(e)$

devine




$L(N) = L(e^*)$

ex: $0^*(01 \cup 10)^*$



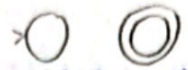

Complexitate:- liniară raportată la lungimea regexului. se parcurge arborele generat de regex.
- numărul de stări din NFA este tot liniar raportat la lungimea regexului.

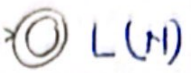
Propoziție: Un NFA N construit dintr-un regex e acceptă același limbaj generat de e.

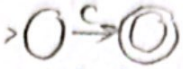


Scanned with CamScanner

$\forall e$, $L(e) = L(N)$

I. caz de bază : • $e = \emptyset$ $\quad L(e) = \emptyset$  $L(N) = \emptyset$

• $e = \varepsilon$ $\quad L(e) = \{\varepsilon\}$  $L(N) = \{\varepsilon\}$

• $e = c$ $\quad L(e) = \{c\}$  $L(N) = \{c\}$

II pasul inductiv

• $e = e_1 e_2$

III. ipoteza inductivă : $L(e_1) = L(N_1)$ și $L(e_2) = L(N_2)$

$\rightarrow L(e_1 e_2) \subseteq L(N)$. $\forall w \in L(e_1 e_2)$ avem $w = w_1 w_2$ a.î. $w_1 \in L(e_1)$ și $w_2 \in L(e_2) \Rightarrow w = w_1 w_2 \in L(N)$

$\rightarrow L(e_1 e_2) \supseteq L(N)$. $\forall w \in L(N)$, din NFA $w_1 \in L(e_1)$ și $w_2 \in L(e_2) \Rightarrow w_1 w_2 \in L(e_1 e_2) \Rightarrow w = w_1 w_2 \in L(e_1 e_2)$

• $e = e_1 \cup e_2$ și $e = e^*$

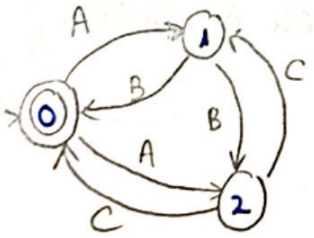
Se demonstrează cu aceeași logică ca mai sus.

$L(e_1 \cup e_2) \subseteq L(N)$ și $L(e_1 \cup e_2) \supseteq L(N)$

$L(e^*) \subseteq L(N)$ și $L(e^*) \supseteq L(N)$

Quiz.

1. Care afirmații sunt adevărate relativ la automatul:






Scanned with CamScanner

- [ ] automatul este determinist
- [x] automatul este nedeterminist
- [x] ABCABC este acceptat
- [x] ABCBCB este acceptat
- [ ] Limbajul $L((A(BC)^*C)^*)$ este inclus în limbajul generat de automat.

- din starea 0 există tranziții pe A în stările 1 și L $\Rightarrow$ $\Rightarrow$ automatul este NFA
- $(0, ABCABC) \vdash_N (1, BCABC) \vdash_N (2, CABC) \vdash_N (0, ABC) \vdash_N \vdash_N (1, BC) \vdash_N (2, C) \vdash_N (0, \varepsilon) \Rightarrow$ ABCABC e acceptat
- $(0, ABCBCB) \vdash_N (1, BCBCB) \vdash_N (2, CBCB) \vdash_N (1, BCB) \vdash_N \vdash_N (2, CB) \vdash_N (1, B) \vdash_N (0, \varepsilon) \Rightarrow$ ABCBCB e acceptat.
- $w = ABCC$; $w \in L((A(BC)^*C)^*)$, dar NFA-ul nu acceptă w $\Rightarrow$ limbajul dat nu e inclus în limbajul generat de automat.

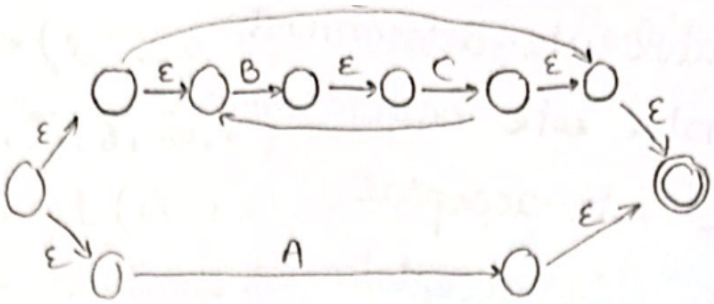
2. Să presupunem că am construit un NFA aplicând algoritmul asupra regexului $AU(BC)^*$. NFA-ul rezultat va avea:

- [ ] 6 stări
- [ ] 7 stări
- [ ] 8 stări
- [ ] 9 stări
- [x] 10 stări
- [ ] 11 stări
- [ ] 12 stări

- [ ] 6 tranziții
- [ ] 7 tranziții
- [ ] 8 tranziții
- [ ] 9 tranziții
- [ ] 10 tranziții
- [ ] 11 tranziții
- [x] 12 tranziții



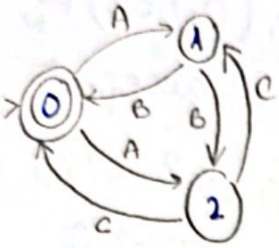
Scanned with CamScanner



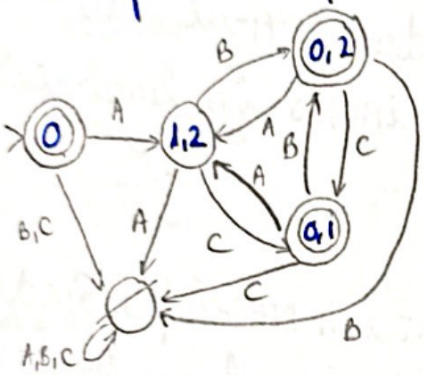

→ 10 stări și 12 tranziții

## Cursul 5 - Transformarea NFA-DFA

Plecând de la NFA-ul:

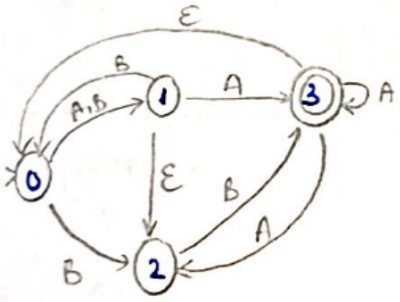



se poate obține următorul DFA:

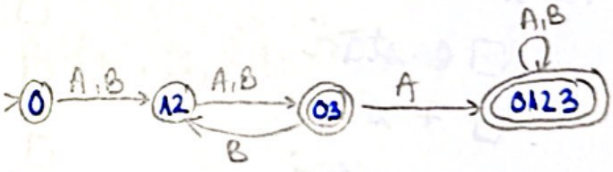



- stările din DFA sunt grupuri de stări din NFA
- acestea simbolizează stările din NFA pe care pot ajunge pe un input dintr-o stare.

Mai este de luat în considerare un element: ε-tranziții.




devine




Când se consideră grupurile de stări, se merge pe toate ε-tranzițiile care pleacă din acele stări.



Scanned with CamScanner

**Definiție:** Fie $N = (K, \Sigma, \Delta, q_0, F)$ un NFA. Construim un DFA $A = (K', \Sigma, \delta, Q_0, F')$ unde:

$K' = 2^K$ → powerset (ex: $K = \{0,1\} \Rightarrow 2^K = \{\emptyset, \{0\}, \{1\}, \{0,1\}\}$)

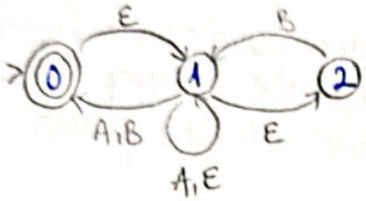
$epsilon_N(q)$ = mulțimea tuturor stărilor din N accesibile din q pe $\varepsilon$-tranziții, inclusiv q.

$\delta(Q, c) = \bigcup_{(q', c, q) \in \Delta,\ q' \in Q} epsilon_N(q)$ → facem epsilon closure pe toți succesorii pe caract c din toate stările din Q pe NFA.

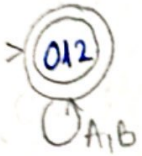
$Q_0 = epsilon_N(q_0)$

$F' = \{Q \in K';\ Q \cap F \neq \emptyset\}$

ex:

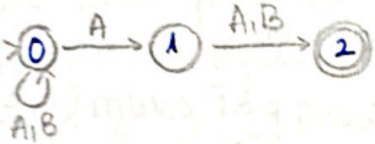



devine:

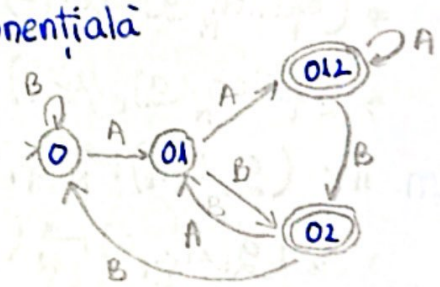



Complexitate - worst-case exponențială

ex:




devine:




Fie $N = (K, \Sigma, \Delta, q_0, F)$ un NFA și $A = (K', \Sigma, \delta, Q_0, F')$ DFA-ul transformat.

**Propoziție:** $L(N) = L(A)$

$\forall w \in \Sigma^* \Rightarrow w \in L(A) \Leftrightarrow w \in L(N)$

$\Rightarrow (Q_0, w) \vdash_A^* (Q_f, \varepsilon), Q_f \in F' \Leftrightarrow (q_0, w) \vdash_N^* (q_f, \varepsilon), q_f \in F$

unei căi din DFA îi corespund mai multe căi din NFA



Scanned with CamScanner

Se demonstrează prin inducție peste $|w|$. Este complicat să demonstrăm o proprietate doar pentru căile „acceptante". În schimb, demonstrăm o proprietate a tuturor căilor

$\forall w \in \Sigma^*$ și toate $q \in K$:

$(epsilon_N(q), w) \vdash_A^* (Q, \varepsilon) \Leftrightarrow (q, w) \vdash_N^* (q_r, \varepsilon) \; ; \; q_r \in Q$

I caz de bază : $|w| = 0 \Rightarrow (epsilon_N(q), \varepsilon) \vdash_A^* (Q, \varepsilon) \Leftrightarrow (q, \varepsilon) \vdash_N^* (q_r, \varepsilon)$, $q_r \in Q$

„$\Rightarrow$" : Avem $Q = epsilon_N(q)$

dacă $q_r \in Q \Rightarrow$ avem $\varepsilon$-tranziții de la $q$ la $q_r$ a.î. $q = q_r$

„$\Leftarrow$" : Avem $\varepsilon$-tranziții din $q$ la orice $q_r \Rightarrow Q = epsilon_N(q)$

II pasul inductiv : $|w| = k \Rightarrow w = c_1 c_2 \dots c_k$

III ipoteza inductivă : $|w| = k-1 \rightarrow$ orice cuvânt de lungime $k-1$ definește o corespondență între căile din DFA și NFA

$w = c_1 c_2 \dots c_{k-1}$

Avem : $(epsilon_N(q), w_{k-1}) \vdash_A^* (Q, \varepsilon) \Leftrightarrow (q, w_{k-1}) \vdash_N^* (q_r, \varepsilon); q_r \in Q$

„$\Rightarrow$" : Presupunem $(epsilon_N(q), u_k) \vdash_A^* (Q, \varepsilon) \Rightarrow$

$\Rightarrow (epsilon_N(q), u_{k-1} c) \vdash_A^* (P, c) \vdash_A (Q, \varepsilon) \Rightarrow$ → s-a introdus o stare suplimentară (penultima)

$\Rightarrow (epsilon_N(q), u_{k-1}) \vdash_A^* (P, \varepsilon)$

din ii : $(q, u_{k-1}) \vdash_N^* (p, \varepsilon)$ pt $\forall p \in P \Rightarrow$

$\Rightarrow (q, u_{k-1} c) \vdash_N^* (p, c)$. $\Rightarrow$ pt un $p \in P$ avem $(p, c) \vdash_w^* (r, \varepsilon)$, cu $r \in Q$

„$\Leftarrow$" : Presupunem $(q, u_k) \vdash_N^* (r, \varepsilon) \Rightarrow$

$\Rightarrow (q, u_{k-1} c) \vdash_N^* (p, c) \vdash_N^* (r, \varepsilon) \Rightarrow$

$\Rightarrow (q, u_{k-1}) \vdash_N^* (p, \varepsilon)$ → același rationament, se adaugă o stare intermediară.

din ii : $(epsilon_N(q), u_{k-1}) \vdash_A^* (p, \varepsilon)$, $p \in P \Rightarrow$

$\Rightarrow (epsilon_N(q), u_{k-1} c) \vdash_A^* (P, c)$

din $(p, c) \vdash_N^* (r, \varepsilon) \Rightarrow (P, c) \vdash_A^* (Q, \varepsilon)$, $r \in Q$

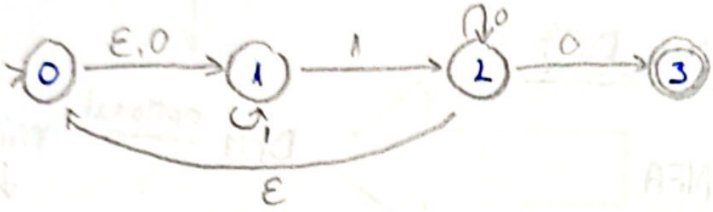


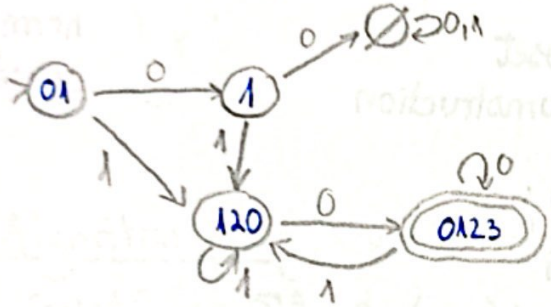
Scanned with CamScanner

Quiz

1. Câte stări accesibile (inclusiv sink states) și stări finale are DFA-ul obținut din transformarea următorului NFA, folosind Subset Construction?

- ☐ 0 stare accesibilă
- ☑ 0 stare finală
- ☐ 2 stări accesibile
- ☐ 2 stări finale
- ☐ 3 stări accesibile
- ☐ 3 stări finale
- ☐ 4 stări accesibile
- ☐ 4 stări finale
- ☑ 5 stări accesibile
- ☐ 5 stări finale
- ☐ 6 stări accesibile
- ☐ 6 stări finale




→ 5 stări accesibile
1 stare finală

2. Câte stări finale și stări accesibile are DFA-ul obținut din transformarea regexului $A(BA)^*$?

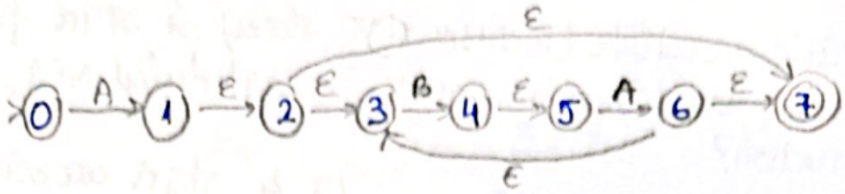
- ☐ 0 stare accesibilă
- ☐ 0 stare finală
- ☐ 2 stări accesibile
- ☑ 2 stări finale
- ☐ 3 stări accesibile
- ☐ 3 stări finale
- ☐ 4 stări accesibile
- ☐ 4 stări finale
- ☑ 5 stări accesibile
- ☐ 5 stări finale
- ☐ 6 stări accesibile
- ☐ 6 stări finale



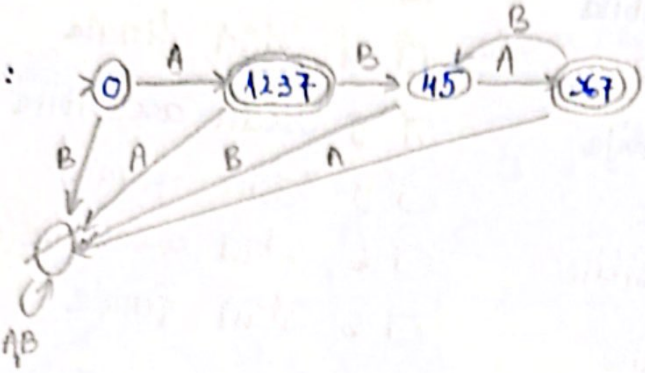
Scanned with CamScanner

$A(BA)^*$

NFA:

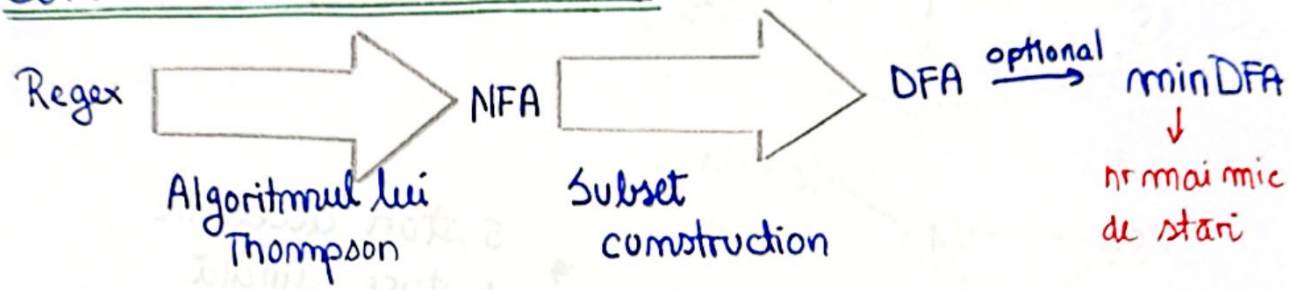



DFA:




⇒ 5 stări accesibile
2 stări finale

## Cursul 6 – Minimizarea DFA

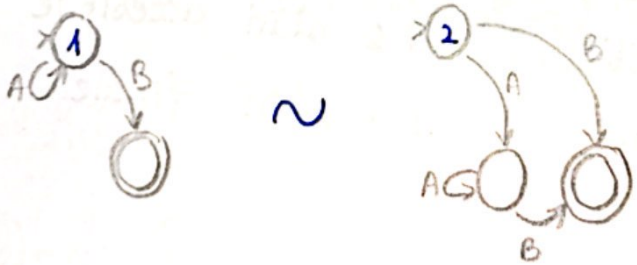



Minimizarea se poate face prin:

- Brzozowsky's algorithm
- Hopcroft's algorithm.

Pentru orice limbaj L, există un unic DFA A a.î. L(A) = L și nu există un alt DFA cu mai puține stări.

Stări distinguishable și indistinguishable

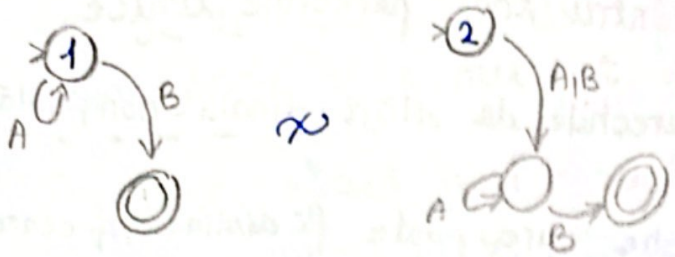



$\forall w \in \Sigma^*, (1, w) \vdash_A (q_f, \varepsilon)$
$\Leftrightarrow (2, w) \vdash_A (q_f', \varepsilon)$, unde
$q_f, q_f' \in F$

Toate cuvintele sunt indistinguishable față de stările 1 și 2.



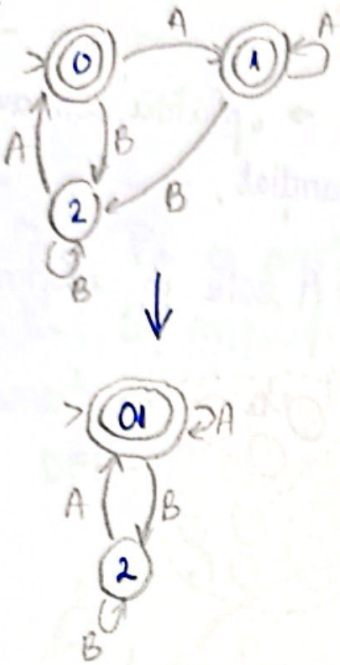
Scanned with CamScanner




$(1,B) \vdash_A (q_f, \varepsilon)$ și
$(2,B) \vdash_A (q, \varepsilon)$, unde
$q_f \in F$ și $q \notin F$

ex: în stânga, cuvântul B este acceptat, în dreapta nu este.

ex:



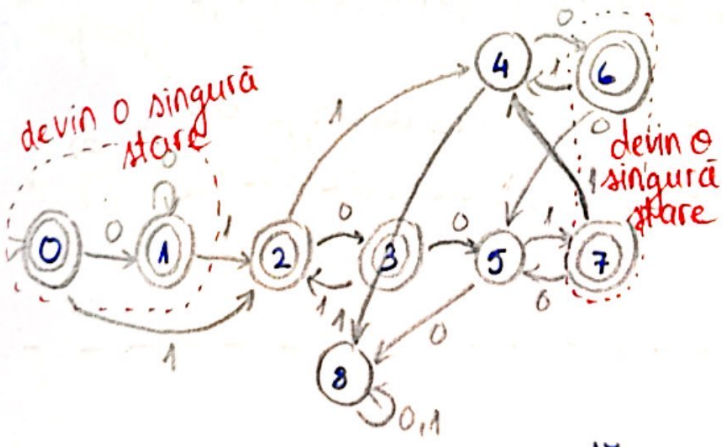

- Stările 0 și 2 se pot distinge (de pe 0 se acceptă $\varepsilon$, de pe 2 nu)
- ——// 1 și 2 ——// ——
- Stările 0 și 1 nu se pot distinge

## Algoritm pt găsirea stărilor distinguishable

Există $w \in \Sigma^*$ a.î. $(x,w) \vdash_A (q_f, \varepsilon)$ și $(y,w) \vdash_A (q, \varepsilon)$, unde $q_f \in F$ și $q \notin F$ sau vice-versa.

Pentru $c \in \Sigma$ și $u, v \in K$, $\delta(c,u) = x$ și $\delta(c,v) = y \Rightarrow$ stările $u$ și $v$ pot fi distinse (cuv cw ce distinge)




| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | x | | | | | | |
| 2 | x | x | | | | | |
| 3 | x | x | x | | | | |
| 4 | x | x | x | x | | | |
| 5 | x | x | x | x | x | | |
| 6 | x | x | x | x | x | x | |
| 7 | x | x | x | x | x | x | x |
| 8 | x | x | x | x | x | x | x |
| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |



Scanned with CamScanner

1. Se crează un tabel pentru toate perechile unice de stări
2. Se marchează toate perechile de stări finală - nonfinală, pt că nu pot fi distinse.
3. Se ia fiecare pereche care poate fi distinsă, fiecare simbol, și se verifică dacă există predecesori care pot fi distinși
4. Se marchează perechile explorate.

---

Complexitate: $O(|K|^2)$ → $\frac{n(n-1)}{2}$ pași → „plătită" în avans când lexerul este instanțiat.

Definiție: O partiție a unei mulțimi A este o ultime $\{P_1, P_2, \dots, P_n\}$ a.î.

- $P_i \subseteq A$
- $\bigcup_i P_i = A$
- $P_i \cap P_j = \emptyset, \forall i \neq j$




Propoziție: Grupând stările care nu pot fi distinse ⇒ ⇒ o partiție pe mulțimea K a stărilor unui DFA.

Demonstrație: Fie $\{P_1, P_2, \dots, P_K\}$ o astfel de partiție. $P_i \subseteq K$ ; $\bigcup P_i = K$. Mai trebuie să demonstrăm că $P_i \cap P_j = \emptyset, \forall i \neq j$ → reducere la absurd.

Fie $P_i$ și $P_j$ două mulțimi de stări care nu pot fi distinse și care au în comun o stare $q$. $i \neq j$ ⇒ un $r \in P_i$ dar $r \notin P_j$

$r \in P_i$ și $q \in P_i$ ⇒ r nu se poate distinge de q
$r \in P_i$ și $q \in P_j$ ⇒ r se poate distinge de q
⇒

⇒ CONTRADICȚIE

---

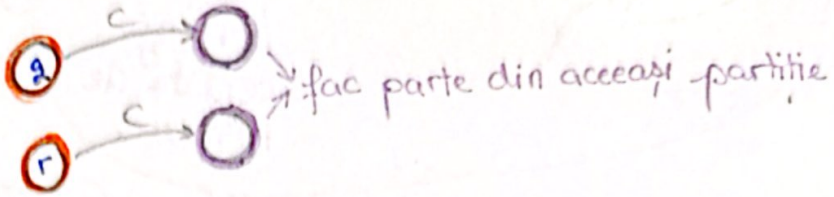


Scanned with CamScanner

5. Se crează stări din fiecare partiție (se grupează doar stările care nu pot fi distinse.

---

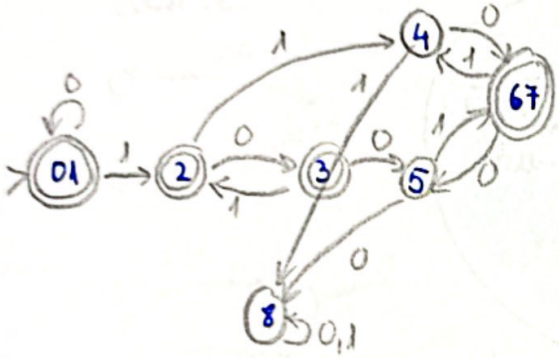
**Propoziție**: Dacă q și r sunt stări care nu pot fi distinse, atunci pt $\forall c \in \Sigma$, $\delta(c,q)$ și $\delta(c,r)$ nu pot fi distinse.




---

Fie $P_i$ o partiție. Pt fiecare $c \in \Sigma (= \{0,1\})$ se alege $q \in P_i$. Se modifică $P_j$ a.î. $\delta(q,c) \in P_j$. Se adaugă tranziții pe c de la $P_i$ la $P_j$.

DFA-ul devine:




- Partiția care conține starea inițială va fi cea inițială.
- Partițiile care conțin stări finale vor fi cele finale.

## Cursul 7 - Limbaje regulate

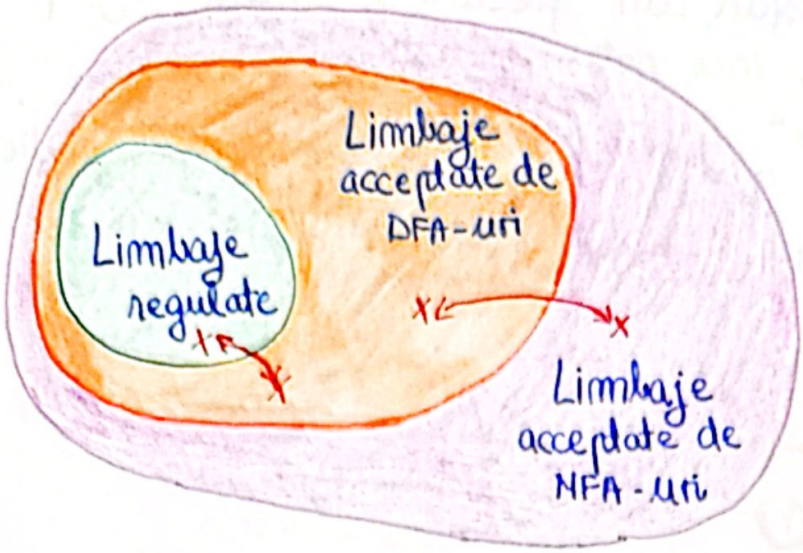
**Definiție**: Un limbaj se numește regulat dacă este generat de o expresie regulată.

**Propoziție**: Dacă un limbaj este regulat, atunci este acceptat de un DFA.

**Propoziție**: Dacă un limbaj este acceptat de un DFA, atunci este acceptat și de un NFA.



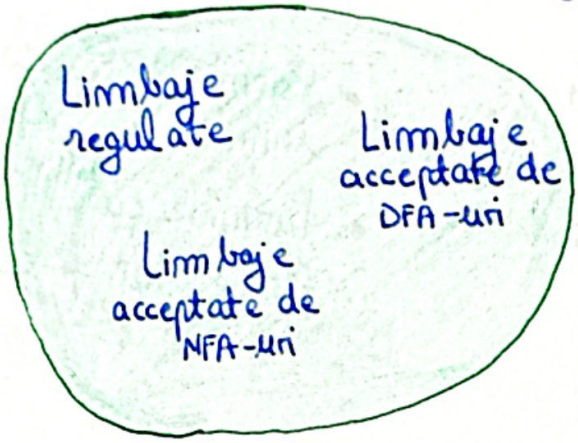
Scanned with CamScanner




**Propoziție**: Dacă un limbaj este acceptat de un NFA, atunci este acceptat de un DFA.

**Propoziție**: Dacă un limbaj este acceptat de un DFA, atunci este regular

$\Rightarrow$




Limbaje regulate
- generatori: Moduri de a descrie limbajul (Regexuri, Gramatici regulate)
- acceptori: Moduri de a verifica apartenența (DFAuri, NFAuri)

## Conversia DFA - Regex

→ se folosește algoritmul State elimination.

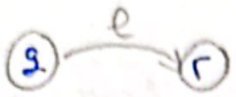
Un NFA era un tuplu $N = (K, \Sigma, \Delta, q_0, F)$ unde

$\Delta \subseteq (K \times \Sigma^* \times K)$ → relația de tranziție.



Scanned with CamScanner

extindem: $\Delta \subseteq (K \times Regexuri \times K)$

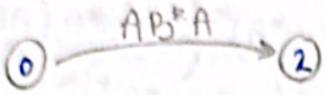



→ din q trebuie consumat un cuvânt generat de e pentru a ajunge în r.

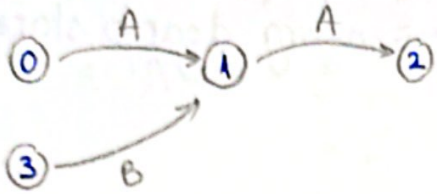
ex:



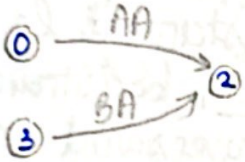

devine:



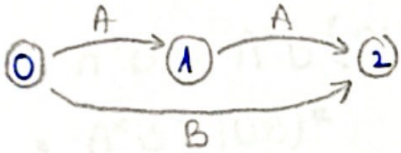

ex:



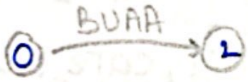

devine:



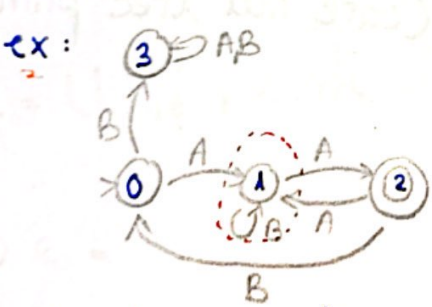

ex:



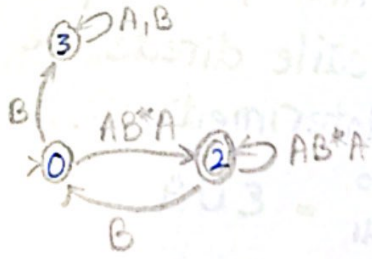

devine:




ex:




devine:




1. Se alege o stare care nu e inițială sau finală (alegem λ).
2. Se identifică toate modurile de a traversa starea, de la predecesori direcți la succesori direcți.



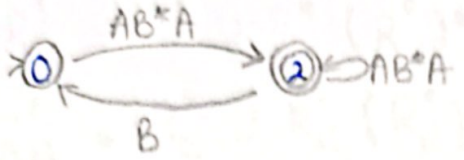
Scanned with CamScanner

- de la 0 la 2 : $AB^*A$
- de la 2 la 2 : $AB^*A$

3. Se elimină starea și se adaugă tranziții pentru fiecare cale identificată.

4. Starea 3 nu are succesori, deci nu există o cale către o stare finală de acolo; o putem elimina.

Rămânem cu:




$\Rightarrow AB^*A\,(AB^*A)^*\,(B\,AB^*A\,(AB^*A)^*)^* =$

$= (AB^*A)^+\,(B\,(AB^*A)^+)^*.$

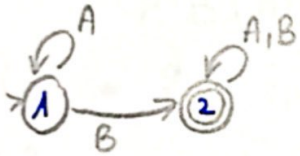
(s-ar repeta procesul până avem doar o stare inițială și stări finale).

Algoritm - idee generală

→ pentru fiecare pereche de stări $(i,j)$ se calculează regexul $R_{ij}$ care ne duce de la starea $i$ la starea $j$.

→ pentru a face procesul iterativ, construim $R_{ij}^k$ care ne duce de la $i$ la $j$ traversând maxim stări $\leq k$

ex:




Pentru fiecare pereche, construim $R_{ij}^0$, care descrie toate căile directe de la $i$ la $j$ (care nu trec prin stări intermediare).

$R_{11}^0 = \varepsilon \cup A$

$R_{12}^0 = B$

$R_{21}^0 = \emptyset$

$R_{22}^0 = \varepsilon \cup A \cup B$



Scanned with CamScanner

Pas inductiv: $R_{ij}^{k} = R_{ij}^{k-1} \cup R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1}$

pt automatul de mai sus:

$R_{11}^1 = R_{11}^0 \cup R_{11}^0 (R_{11}^0)^* R_{11}^0 =$

$= \varepsilon \cup A \cup (\varepsilon \cup A)(\varepsilon \cup A)^* (\varepsilon \cup A) = A^*$

$R_{12}^1 = R_{12}^0 \cup R_{11}^0 (R_{11}^0)^* R_{12}^0 =$

$= B \cup (\varepsilon \cup A)(\varepsilon \cup A)^* B = B \cup A^* B = A^* B$

$R_{21}^1 = R_{21}^0 \cup R_{21}^0 (R_{11}^0)^* R_{11}^0 = \emptyset$

$R_{22}^1 = R_{22}^0 \cup R_{21}^0 (R_{11}^0)^* R_{12}^0 =$

$= \varepsilon \cup A \cup B \cup \underbrace{\emptyset \cdot \ldots}_{= \emptyset} = \varepsilon \cup A \cup B$

$R_{11}^2 = R_{11}^1 \cup R_{12}^1 (R_{22}^1)^* R_{21}^1 =$

$= A^* \cup A^* B (\varepsilon \cup A \cup B) \cdot \emptyset = A^*$

$R_{12}^2 = R_{12}^1 \cup R_{12}^1 (R_{22}^1)^* R_{22}^1 =$

$= A^* B \cup A^* B (\varepsilon \cup A \cup B)^* (\varepsilon \cup A \cup B) =$

$= A^* B (A \cup B)^*$

$R_{21}^2 = \emptyset$

$R_{22}^2 = (\varepsilon \cup A \cup B)^*$

$e = \bigcup R_{1f}^n$, unde $n$ = nr total de stări
$f$ = o stare finală

$\Rightarrow e = R_{12}^2 = A^* B (A \cup B)^*$



Scanned with CamScanner

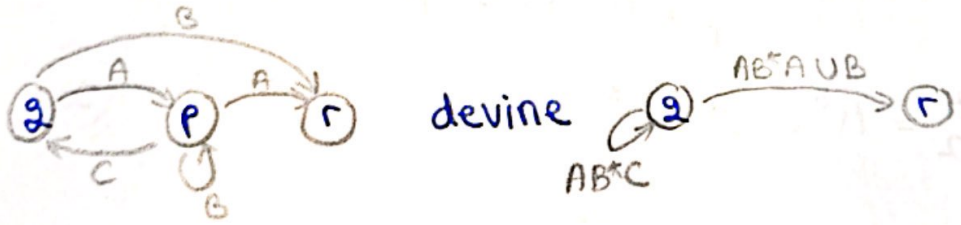
## Quiz

1. Fie $L_1$ un limbaj acceptat de un NFA și $L_2$ un limbaj acceptat de o expresie regulată ce nu conține operatorul de reuniune. Care afirmații sunt corecte?

- [x] $L_1$ este un limbaj regulat
- [x] $L_2$ este un limbaj regulat
- [x] Există un DFA care acceptă $L_2$
- [ ] Toate NFA-urile care acceptă $L_2$ nu conțin $\varepsilon$-tranziții
- [x] Reuniunea limbajelor $L_1$ și $L_2$ este un limbaj regulat
- [x] Există o expresie regulată care acceptă concatenarea lui $L_1$ cu $L_2$.

2. Care din tranzițiile de mai jos vor fi adăugate în automatul de mai jos, în ipoteza că starea p va fi eliminată?

- [ ] $(q, AB^*, p)$
- [ ] $(q, AB^*C, r)$
- [x] $(q, AB^*A \cup B, r)$
- [ ] $(q, AB^+A \cup B, r)$
- [ ] $(p, B^*A, r)$
- [ ] $(q, AA, r)$
- [ ] $(q, ACB, r)$
- [ ] $(p, CB, r)$
- [ ] $(q, AA \cup B, r)$






Scanned with CamScanner

# Cursul 8 - Closure Properties

→ o varietate de transformări peste un limbaj îi păstrează regularitatea

→ o varietate de combinații de limbaje își păstrează regularitatea

**Propoziție:** Concatenarea a două limbaje regulate este un limbaj regulat.

**Propoziție:** Reuniunea a două limbaje regulate este un limbaj regulat.

**Propoziție:** Kleene-Star-ul unui limbaj regulat este un limbaj regulat.

**Propoziție:** Complementul unui limbaj regulat este un limbaj regulat.

**Propoziție:** Inversarea unui limbaj regulat este un limbaj regulat

$$inverse(L) = \{ w^R \mid w \in L \}$$

↳ cuvântul w inversat

**Propoziție:** Intersecția a două limbaje regulate este un limbaj regulat.

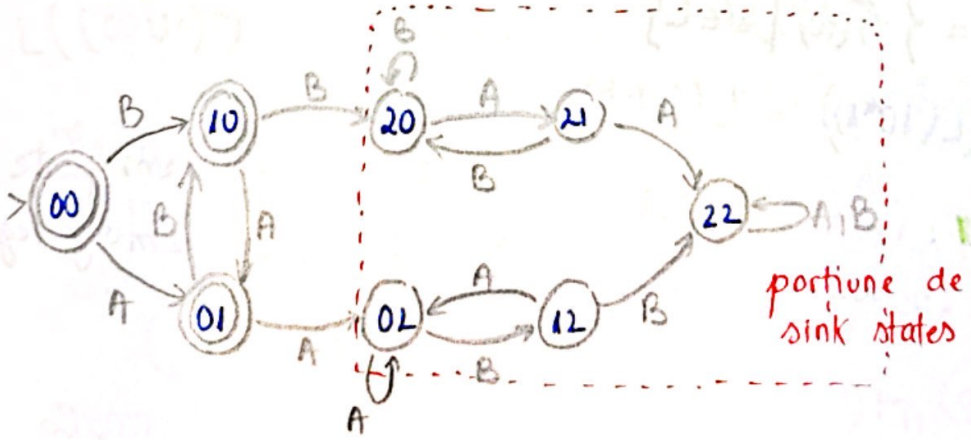
ex:




acceptă cuvinte fără B consecutivi




acceptă cuvinte fără A consecutivi.






Scanned with CamScanner

**Demonstrație:** Fie $A_1 = (K_1, \Sigma, \delta_1, q_0^1, F_1)$ și $A_2 = (K_2, \Sigma, \delta_2, q_0^2, F_2)$ care acceptă limbajele $L_1$ și $L_2$.

Construim automatul $A_x$ care acceptă $L_1 \cap L_2 \Rightarrow$

$\Rightarrow A_x = (K_1 \times K_2, \Sigma, \delta_x, (q_0^1, q_0^2), F_1 \times F_2)$, unde:

$\delta_x((q_1, q_2), c) = (q_1', q_2')$ dacă $\delta_1(q_1, c) = q_1'$ și $\delta_2(q_2, c) = q_2'$

Arătăm că: $((q_0^1, q_0^2), w) \vdash_A^* ((q_f^1, q_f^2), \varepsilon)$, unde $(q_f^1, q_f^2) \in F_1 \times F_2$. $\Leftrightarrow$

$\Leftrightarrow (q_0^1, w) \vdash_{A_1}^* (q_f^1, \varepsilon)$, $q_f^1 \in F_1$ și $(q_0^2, w) \vdash_{A_2}^* (q_f^2, \varepsilon)$, $q_f^2 \in F_2$

$\Rightarrow$ reiese din construcția automatului.

**Propoziție.** Diferența a două limbaje regulate este un limbaj regulat.
↳ toate cuv din $L_1$ care nu sunt în $L_2$

## Homomorphism

$h : \Sigma \to \Sigma^*$ ; $h(0) = ab$ ; $h(1) = \varepsilon$

($\Sigma$: simbol, $\Sigma^*$: cuvânt)

Extindem $h$ peste cuvinte:

$h(c_1 \dots c_n) = h(c_1) \dots h(c_n)$ ; $h(0010) = h(0)h(0)h(1)h(0) = abababab$

Extindem $h$ peste limbaje:

$h(L) = \{ h(w) \mid w \in L \}$

ex: $h(L(10^*1)) = L((ab)^*)$

**Propoziție:** Dacă aplicăm un homomorphism peste un limbaj regulat, obținem un limbaj regulat.



Scanned with CamScanner

**Demonstrație:** Fie $E$ expresia regulată care generează limbajul $L$ și $h$ un homomorfism. Fie $E^h$ expresia regulată obținută prin înlocuirea fiecărui simbol $c$ din $E$ cu $h(c) \Rightarrow E^h$ generează $h(L)$

## Homomorfisme inverse

$h: \Sigma \to \Sigma^*$ ; $h(0) = ab$ ; $h(1) = \varepsilon$

$h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L\}$

ex: $h^{-1}(L(\varepsilon)) = L(1^*)$

$h^{-1}(L((ab)^*)) = L((1 \cup 0)^*)$

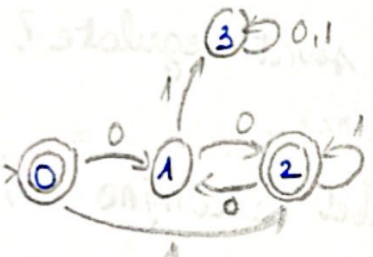
$h^{-1}(L(a^*b^*)) = \{1, 0\} = L(1^*(0 \cup \varepsilon)1^*)$

$h(a) = 01$, $h(b) = 10$ $\Rightarrow$

$\Rightarrow h^{-1}(L((00 \cup 1)^*)) = L((ba)^*)$

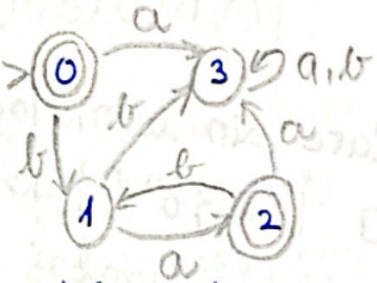
$h^{-1}(L((1 \cup 0)^*)) = L((a \cup b)^*)$

**Propoziție:** Aplicând un homomorfism invers unui limbaj regulat obținem un limbaj regulat.




$L((00 \cup 1)^*)$

$h(a) = 01$
$h(b) = 10$
$\Longrightarrow$




$L((ba)^*)$

Fie $A$ un DFA care acceptă $L$ și $h$ un homomorfism. Construim un DFA cu aceleași stări ca $A$.

Pt fiecare stare $q$ și $w$ a.î. $h(c) = w$ fie $(q, w) \vdash_A (q', \varepsilon)$. se adaugă o tranziție din $q$ în $q'$ pe $c$

Dem că $(q_0, h(w)) \vdash (q_f, \varepsilon) \Leftrightarrow (q_0, w) \vdash_H (q_f, \varepsilon)$.



Scanned with CamScanner

**Propoziție:** Fie $A = (K, \Sigma, \delta, q_0, F)$ un DFA. Fie $L$ limbajul care conține cuvinte acceptate de $A$, care vizitează fiecare stare din $A$ cel puțin o dată. Atunci, limbajul $L$ este regulat.

## Quiz

1. Care din următoarele limbaje sunt regulate?
   - ☑ $L(0^*1^*)$
   - ☑ $L((01)^*)L(N)$, unde $N$ este un NFA cu o singură stare finală.
   - ☑ $L(A)L(N)$, unde $A$ este un DFA și $N$ un NFA
   - ☐ $L(A)L$, unde $A$ este un DFA și $L$ este limbajul peste $\{0,1\}$ în care nr de 0-uri este egal cu nr de 1-uri.
   - ☑ complement$(L)$, unde $L$ este un limbaj finit.
   - ☑ diferența simetrică între două limbaje acceptate de NFA-uri.
     → $(A \setminus B) \cup (B \setminus A)$

2. Care din următoarele limbaje sunt regulate?
   - ☐ $L = \{0^n 1^n \mid n > 0\}$
   - ☐ orice limbaj peste un alfabet ce conține un singur simbol
   - ☑ intersecția a două limbaje (nu neapărat regulate) peste alfabete ce nu au simboli în comun
   - ☑ limbajul peste alfabetul $\{0\}$, ce conține șiruri ale căror lungime este divizibilă cu 7.



Scanned with CamScanner

3. Care afirmații sunt adevărate?

☑ Există un limbaj L care, reunit cu un limbaj care nu e regulat, produce un limbaj regulat.

☐ Orice limbaj L, reunit cu un limbaj care nu e regulat, produce un limbaj regulat.

☑ Există un limbaj L care, concatenat cu un limbaj care nu e regulat, produce un limbaj regulat.

☐ Orice limbaj L, concatenat cu un limbaj care nu e regulat, produce un limbaj regulat

☐ Complementul unui limbaj regulat este un limbaj neregulat.

## Cursul 9 - The pumping lemma

Avem expresii regulate de forma:

E ::= E U E | EE | E* | ID | ε where ID = F | C

F ::= F U F | FF | F* | C

ID ::= <string>

ex: E = ST (a ∪ b)* ST where ST = a⁺ ≠ a⁺(a ∪ b)* a⁺

w ∈ L(E) ⇔ w = xyx cu x ∈ L(a⁺) și y ∈ L((a ∪ b)*)

Pentru orice expresie E definită ca mai sus, L(E) nu este un limbaj regulat.

ex: E = ST b* ST where ST = a⁺

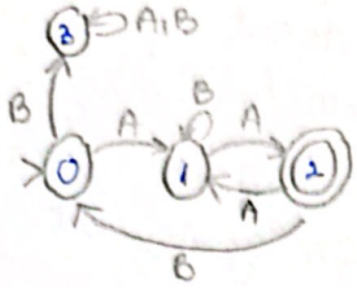
L(E) = {aⁿ bᵐ aⁿ | n, m ≥ 0} → acceptarea cuvintelor necesită numărarea caract, nu se poate face cu automate



Scanned with CamScanner

Lemma de pompare este o proprietate care descrie limbaje regulate fără să vorbească despre generatori și acceptori.

Când un limbaj nu are această proprietate (sau respectă complementul ei), limbajul nu este regulat.




Presupunem că alegem un cuvânt foarte lung din $L(A)$

$AAAAAA \in L(A)$

$ABBBBAAAAA \in L(A)$

$ABBBBAABBBBAAA \in L(A)$

⇒ obținem tot felul de repetiții

**Pumping Lemma**: Toate cuvintele lungi dintr-un limbaj regulat sunt formate din repetiții ale unui fragment

**Repetiție**: Fie $L$ un limbaj regulat infinit. Există un număr $n$ a.î. pt fiecare cuvânt $w \in L$, $|w| \geq n$, $w = xyz$, unde

- $|xy| \leq n$
- $y \neq \varepsilon$
- pt toți $k \geq 0$, avem $xy^kz \in L$

ex: (pt automatul de mai sus)

$ABBBBAAAAA \in L(A)$ : $x = A, y = B, z = AAAAA \Rightarrow xy^2z \in L, xy^3z \in L, \ldots$

$ABABABBA \in L(A)$ : $x = \varepsilon, y = ABA, z = BABBA \Rightarrow xy^2z \in L, \ldots$

**Demonstrație**: Fie $A$ un DFA care acceptă $L$. Alegem $n = |K|$. Alegem $w \in L$, $|w| \geq n$.

Avem $(q_0, w) \vdash^* (q_f, \varepsilon)$. Datorită lungimii cuv, cel puțin o stare este vizitată de 2 ori.

$\Rightarrow (q_0, w) \vdash^* (q_i, ?) \vdash^* (q_i, ?) \vdash^* (q_f, \varepsilon)$



Scanned with CamScanner

Presupunem că starea q a doua este prima revizitare a stării q.

$(q_0, xyz) \vdash^* (q, yz) \vdash^* (q, z) \vdash^* (q_f, \varepsilon)$

până aici toate stările vizitate sunt unice $\Rightarrow |xy| \leq |K|$ ✓

q este vizitat de cel puțin 2 ori $\Rightarrow y \neq \varepsilon$ ✓

$(q_0, xy^K z) \vdash^* (q, y^K z) \vdash^* (q, y^{K-1} z) \vdash^* \dots \vdash^* (q, yz) \vdash^* (q, z) \vdash^* (q_f, \varepsilon)$

$\Rightarrow$ pt $\forall K > 0$, $xy^K z \in L$ ✓

Structură: Dacă L este regulat, atunci L are proprietatea de pompare.

$A \Rightarrow B \xrightarrow[\text{negăm}]{\text{dacă}}$ not B $\Rightarrow$ not A $\Rightarrow$

$\Rightarrow$ Complementul Lemmei de Pompare: Dacă L nu are proprietatea de pompare, atunci L nu este regulat.

Fie L un limbaj a.î. pt $\forall n$ există $w_n$ a.î. indiferent cum am împărți w în xyz, unde

$|xy| \leq n$;

$y \neq \varepsilon$,

există $K \geq 0$ a.î. $xy^K z \notin L$, atunci limbajul nu este regulat.

Aplicații

1. $L(e) = \{A^n B^m A^n \mid n, m > 0\}$

Fixăm $w_n = A^n B A^n$ → alegere liberă

AAAA ... AAAAAAAAABAA ... AAAAAAA

x y z

$x = A^q$; $y = A^r$; $z = A^p B A^n$, cu $q + r + p = n$ și $r > 0$

Alegem $K = 0 \Rightarrow xy^K z = A^{q+p} B A^n$, dar $q + p < n \Rightarrow xy^K z \notin L$



Scanned with CamScanner

EXPR ::= EXPR+EXPR | EXPR*EXPR | (EXPR) | NUMBER

$(1+2^*(4+5)^*(5+(7^*8))) \in L_{EXPR}$ ; $(((((0)))))\in L_{EXPR}$

→ este neregulat deoarece trebuie numărate parantezele

2) $L_{EQ} = \{A^nB^n \mid n \geq 0\}$

Fixăm $w_n = A^nB^n$

AAAAA.... AAAAAABBBB .... BBBB

x y z

$x = A^q$, $y = A^r$, $z = A^pB^n$, unde $q+r+p = n$ și $r>0$

Alegem $k=0 \Rightarrow xy^kz = A^{q+p}B^n$, dar $q+p<n \Rightarrow xy^kz \notin L$

$L_{EQ}$ poate lua forma $\{(^n0)^n \mid n \geq 0\} \Rightarrow L_{EQ} \subseteq L_{EXPR}$

!! Dacă $L_{EQ}$ nu e regulat și $L_{EQ} \subseteq L_{EXPR} \not\Rightarrow L_{EXPR}$ nu e regulat

3) $L_{prime} = \{0^n \mid n$ este nr prim$\}$

Fixăm $w_n = 0^p$, p este primul nr prim mai mare ca n

$w_m = xyz = 0^\alpha 0^\beta 0^\gamma$, $\alpha+\beta+\gamma = 0$, $\beta \neq 0$

Presupunem $\alpha+\gamma > 0$. Fixăm $k = \alpha+\gamma$

$\Rightarrow w_n = xy^kz = 0^\alpha 0^{k\beta} 0^\gamma = 0^{\alpha+(\alpha+\gamma)\beta+\gamma} = 0^{\overbrace{(\alpha+\gamma)(\beta+1)}^{\text{nu e prim}}} \notin L_{prime}$

## Cursul 10 - Push-down automata (PDA)

→ acceptori pentru context-free languages

→ se mai cheamă automate cu stivă. → se poate accesa doar primul element.



Scanned with CamScanner

# Context-free languages

$L_1$ : E ::= E U E | E E | E* | ID | ( E ) where ID = F | C
F ::= F U F | F F | F* | C
ID ::= <string>
Not context-free

$L_2 = \{ a^n b^m a^n \mid n, m \geq 0 \}$ Context-free

$L_3 = \{ 0^n \mid n$ e un nr prim$\}$ Context-free

$L_4$ : EXPR ::= EXPR + EXPR | EXPR * EXPR | (EXPR) | NUMBER
Not context-free

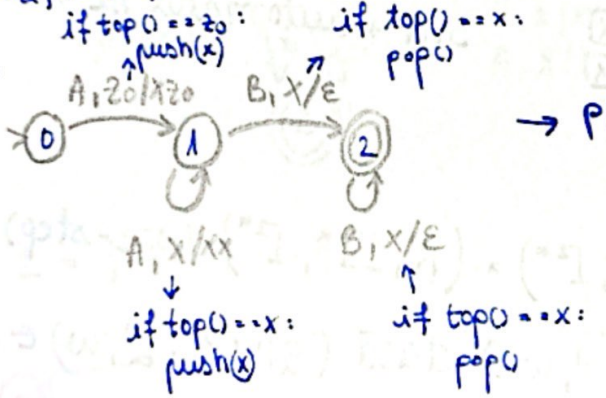
# Configuratiile unui PDA

$(q, w, \gamma) \in K \times \Sigma^* \times \Gamma^*$ → alfabetul stivei

PDA-ul este în starea $q$, cuvântul este $w$ și stiva curentă este $\gamma$.

ex: $(q_0, AABB, Z_0) \vdash (?, ABB, XZ_0) \vdash (?, BB, XXZ_0) \vdash (?, B, XZ_0) \vdash$
$\vdash (q_f, \varepsilon, Z_0)$

ex:




→ PDA pt $A^n B^n$

$(0, AABB, Z_0) \vdash (1, ABB, XZ_0) \vdash (1, BB, XXZ_0) \vdash (2, B, XZ_0) \vdash (2, \varepsilon, Z_0)$

$(0, AAB, Z_0) \vdash (1, AB, XZ_0) \vdash (1, B, XXZ_0) \vdash (2, \varepsilon, XZ_0)$ → am rămas pe stivă cu ceva

$(0, AABBA, Z_0) \vdash (1, AABBA, XZ_0) \vdash (1, ABBA, XXZ_0) \vdash (1, BBA, XXXZ_0) \vdash$
$\vdash (2, BA, XXZ_0) \vdash (2, A, XZ_0)$ → se blochează automatul dacă nu avem o tranziție lună



Scanned with CamScanner

**Definiție**: Un PDA este tuplul $A = (K, \Sigma, \Gamma, \Delta, q_0, F)$ unde

$K$ = mulțimea stărilor

$\Sigma$ = alfabetul de input

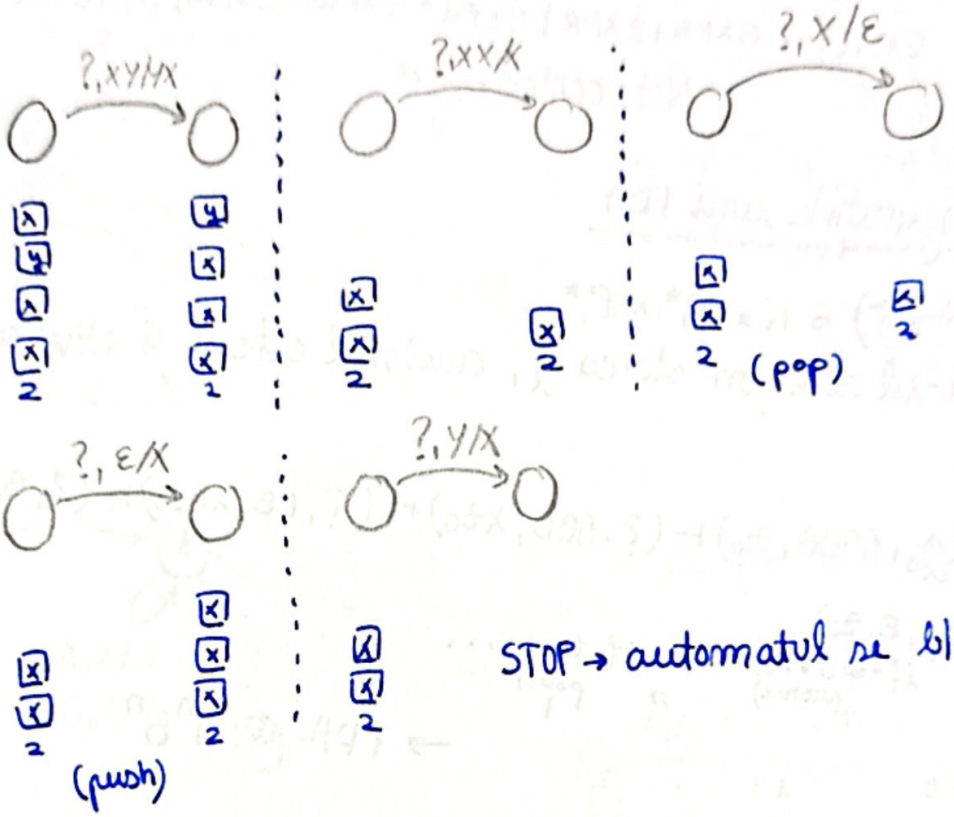
$\Gamma$ - alfabetul stivei

$\Delta \subseteq (K \times \Sigma^* \times \Gamma^* \times K \times \Gamma^*)$ = relația de tranziție

$q_0$ - starea inițială

$F$ - mulțimea stărilor finale

ex:




STOP → automatul se blochează.

Relația $\vdash_A \subseteq (K, \Sigma^*, \Gamma^*) \times (K, \Sigma^*, \Gamma^*)$ (one-step)

$(q, ww', \gamma\alpha) \vdash_A (q', w', w\alpha)$ dacă $(q, w, \gamma, q', w) \in \Delta$

Acceptare din starea finală:

Un cuvânt w este acceptat de A dacă $(q_0, w, z_0) \vdash_A^* (q, \varepsilon, \gamma)$ și $q \in F$. → cuv consumat, nu contează ce este pe stivă.

Acceptare prin stivă vidă:

Un cuvânt w este acceptat de A dacă $(q_0, w, z_0) \vdash_A^* (q, \varepsilon, \varepsilon)$ → cuv consumat și stiva goală.

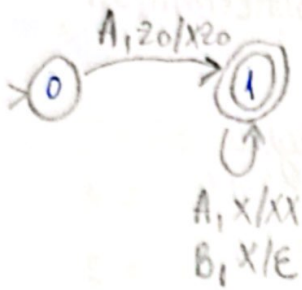


Scanned with CamScanner

**Propoziție**: Un limbaj este acceptat de un PDA prin stare finală dacă este acceptat de un PDA prin stivă vidă.

ex: $L = \{w \in \{A,B\}^* \mid \#_A(w) = \#_B(w)\}$

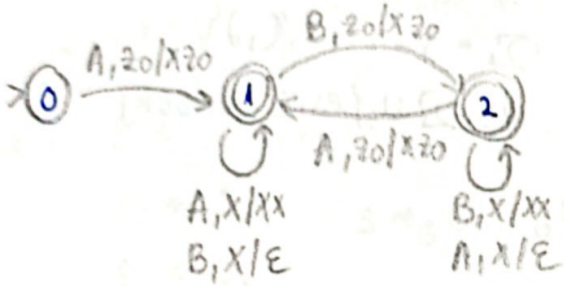
↳ nr de caract A din w




$(0, AABBAB, Z_0) \vdash_A (1, ABBAB, XZ_0) \vdash_A (1, BBAB, XXZ_0)$

$\vdash_A (1, BAB, XZ_0) \vdash_A (1, AB, Z_0) \rightarrow$ STOP, nu avem tranziție

problemă similară pt AABBBA.




$(0, AABBBA) \vdash_A \dots \vdash_A (1, BA, Z_0) \vdash_A$

$\vdash_A (2, A, XZ_0) \vdash_A (2, \varepsilon, Z_0)$

$(0, AAA, Z_0) \vdash_A^* (1, \varepsilon, XXXZ_0)$

cuv se acceptă dar nu e ok.




→ acum este ok.

ex: $L = \{ww^R \mid w \in \{A,B\}^*\}$




$(0, AABBAA, Z_0) \vdash_A^* (0, BAA, BAAZ_0) \vdash_A$

$\vdash_A (0, AA, BBAAZ_0)$ stuck

sau

$\vdash_A (1, BAA, BAAZ_0) \vdash_A^* (1, \varepsilon, Z_0) \vdash_A (2, \varepsilon, \varepsilon)$



Scanned with CamScanner

# Cursul 11 - Context-free grammars

Forma Backus-Naur (BNF)

EXPR ::= EXPR + EXPR | EXPR * EXPR | (EXPR) | NUMBER

**Definiție**: Un context-free grammar este un tuplu $G = (V, \Sigma, R, S)$, unde:

$V$ = o mulțime de terminali și neterminali

$\Sigma$ = alfabetul

$R \subseteq V \setminus \Sigma \times V^*$ - mulțimea de reguli

$S$ = simbolul de start.

ex: $S \leftarrow EXPR$

$EXPR \leftarrow EXPR + EXPR$

$EXPR \leftarrow EXPR * EXPR$

$EXPR \leftarrow (EXPR)$

$EXPR \leftarrow NUMBER$

$NUMBER \leftarrow ONUMBER$

$NUMBER \leftarrow 1NUMBER$

$NUMBER \leftarrow \varepsilon$

$\Sigma = \{0, 1, +, *, (, )\}$

$V = \Sigma \cup \{EXPR, NUMBER\}$

**Definiție**: Fie $G$ o gramatică, $\alpha A \beta$ și $\alpha \gamma \beta$ stringuri din $V^*$ și fie $A \leftarrow \gamma$ o regulă din $G$. Atunci $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$ ($\alpha \gamma \beta$ derivăm într-un pas $\alpha A \beta$)

ex: (de mai sus)

$S \Rightarrow_G EXPR \Rightarrow_G EXPR + EXPR \Rightarrow_G (EXPR) + EXPR \Rightarrow_G (EXPR) + NUMBER \Rightarrow_G$
$\Rightarrow_G (EXPR * EXPR) + NUMBER \Rightarrow_G (EXPR * EXPR) + ONUMBER \Rightarrow_G \dots$

**Definiție**: Fie $G$ o gramatică, $\alpha$ și $\beta$ șiruri din $V^*$. Spunem că $\alpha$ poate fi derivat din $\beta$ în zero sau mai multi pași, $\alpha \Rightarrow_G^* \beta$ dacă $\alpha = \beta$ sau $\alpha \Rightarrow_G \alpha_1 \Rightarrow_G \alpha_2 \Rightarrow_G \dots \Rightarrow_G \beta$.



Scanned with CamScanner

Definitie: Fie un context-free grammar G. Limbajul L(G) acceptat este: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$

ex: $S \leftarrow aT \mid \varepsilon$
$T \leftarrow Sb$

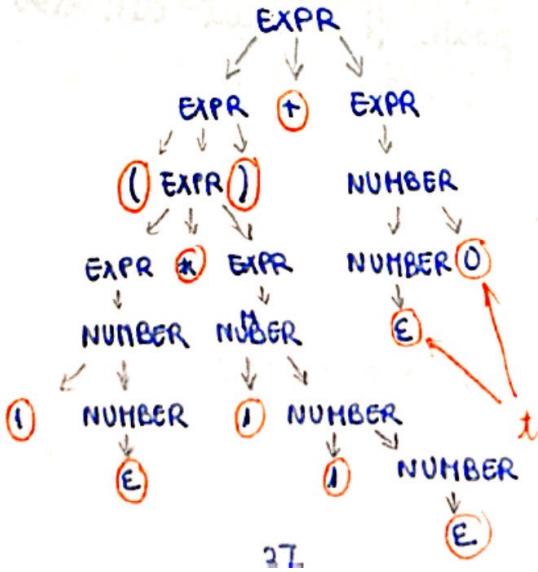
$S \Rightarrow_G aT \Rightarrow_G aSb \Rightarrow_G aaTb \Rightarrow_G aaSbb \Rightarrow_G aabb \Rightarrow$
$\Rightarrow L(G) = \{a^n b^n \mid n \geq 0\}$

ex: $S \leftarrow AB$
$A \leftarrow aAb \mid \varepsilon$
$B \leftarrow bBc \mid \varepsilon$
$\Rightarrow L(G) = \{a^n b^{n+m} c^m \mid n, m \geq 0\}$

ex: $S \leftarrow 0A \mid 1B \mid \varepsilon$
$A \leftarrow S0$
$B \leftarrow S1$
$S \Rightarrow_G 0A \Rightarrow_G 0S0 \Rightarrow_G 01B0 \Rightarrow_G 01S10 \Rightarrow_G \dots$
$\Rightarrow L(G) = \{ww^R \mid w \in \{0,1\}^*\}$
cuvinte liniare palindroame

## Arbore de parsare

$S \Rightarrow_G EXPR \Rightarrow_G EXPR + EXPR \Rightarrow_G (EXPR) + EXPR \Rightarrow_G (EXPR) + NUMBER \Rightarrow_G$
$\Rightarrow_G (EXPR * EXPR) + NUMBER \Rightarrow_G (EXPR * EXPR) + 0NUMBER \Rightarrow_G (NUMBER *$
$EXPR) + 0NUMBER \Rightarrow_G (1NUMBER * EXPR) + 0 \Rightarrow_G (1 * EXPR) + 0 \Rightarrow_G$
$\Rightarrow_G (1 * NUMBER) + 0 \Rightarrow_G (1 * 11NUMBER) + 0 \Rightarrow_G (1 * 11) + 0.$




$\Rightarrow (1 * 11) + 0$



Scanned with CamScanner

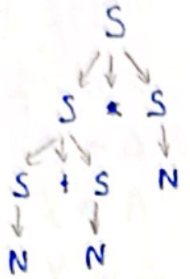
Când implementăm un parser → Abstract Syntax Trees (AST)

## Ambiguitate

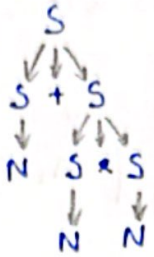
S ← S+S | S*S | (S) | N

N ← ON | IN | ε

pt 0+1*1 :




sau




⇒ are 2 arbori diferiți ⇒ ⇒ este ambiguu.

**Definiție**: O derivare $\alpha A\beta \Rightarrow_G \alpha\gamma\beta$ se numește left-most dacă : - $\alpha$ conține doar simboli din alfabet
- A este primul non-terminal de la stânga

**Propoziție** : O secvență de derivări left-most determină în mod unic un arbore de parsare.

**Definiție**: O gramatică independentă de context este ambiguă dacă există un cuvânt care :

a) poate fi derivat din două derivări left-most diferite SAU

b) poate fi obținut din doi arbori de parsare diferiți.

## Soluție pt $L_{EXPR}$

S ← M+S | M

M ← A*M | A

A ← (S) | N

N ← ON | IN | ε

→ oferă precedență



Scanned with CamScanner

**Proprietate**: Există limbaje independente de context care nu pot fi generate de gramatici neambigue.

**Proprietate**: Problema de decidere dacă o gramatică este ambiguă este nedecidabilă ⇒ nu există un algoritm care să o rezolve.

## Cursul 12 - Gramatici regulate. Forma normală Chomsky.

O gramatică este regulată dacă toate regulile ei sunt de forma:

$X \leftarrow aA$
$X \leftarrow A$
$X \leftarrow a$
$X \leftarrow \varepsilon$

sau

$X \leftarrow Aa$
$X \leftarrow A$
$X \leftarrow a$
$X \leftarrow \varepsilon$

**Propoziție**: Gramaticile regulate generează limbaje regulate.

**Definiție**: O gramatică este în forma normală Chomsky dacă fiecare regulă a ei are una din formele:

- $X \leftarrow YZ$ → neterminali
- $X \leftarrow a$ → terminal

→ generează arbori binari completi

**Propoziție**: Fie $L$ un limbaj independent de context a.î. $L \neq \emptyset$ și $\varepsilon \notin L$. Există o gramatică CNF $G$ a.î. $L(G) = L$



Scanned with CamScanner

## Quiz

1. Care din următoarele limbaje sunt regulate?
   - ☐ $L(G)$, unde $G$ este definită: $S \leftarrow aA \mid Bb \mid a$, $A \leftarrow Sb \mid a$, $B \leftarrow aS \mid b$
   - ☑ $L(G)$, unde $G$ este definită: $S \leftarrow aS \mid A$, $A \leftarrow aS \mid bC$, $C \leftarrow bS \mid aA \mid a$
   - ☐ $L(A)$, unde $A$ e un PDA care folosește cel mult $n$ poziții de pe stivă, pt orice cuv de lungime $n$.
   - ☑ $L(A)$, unde $A$ e un PDA care folosește cel mult $n$ poziții de pe stivă.
   - ☑ $L(N) \cap L(A)$, unde $N$ este un NFA și $A$ e un DFA
   - ☐ $L(G)$, unde $G$ este o gramatică CNF.

**Propoziție:** Mulțimea limbajelor generate de un context-free grammar este exact cea acceptată de PDA-uri.

## Cursul 13 - Closure properties for CF languages

**Propoziție:** Intersecția a două limbaje independente de context nu este neapărat independentă de context.

**Propoziție:** Complementul unui limbaj CF nu este neapărat CF.

**Propoziție:** Diferența a două limbaje CF nu este neapărat CF.



Scanned with CamScanner

**Propoziție**: Concatenarea a două limbaje CF este CF

**Propoziție**: Reuniunea a două limbaje CF este CF

**Propoziție**: Kleene-Star-ul unui limbaj CF este CF

**Propoziție**: Inversul unui limbaj CF este CF

**Propoziție**: Intersecția între un limbaj CF și un limbaj regulat este CF.

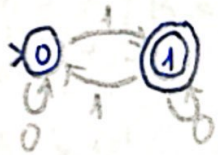
**Propoziție**: Diferența între un limbaj CF și unul regulat este CF.

# Laboratoare

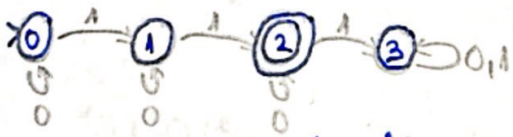
## Laboratorul 1

1. Scrieți un DFA pentru:

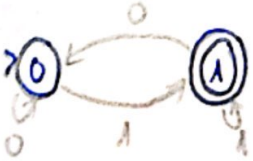
1.1. $L = \{w \in \{0,1\}^* \mid w \text{ conține un nr impar de } 1\}$



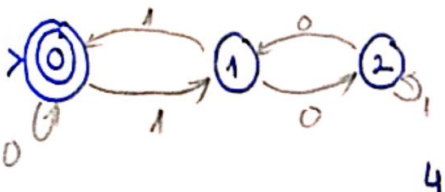

1.2. Limbajul cuv. liniare care conțin exact 2 de 1.




1.3. Limbajul cuvintelor liniare impare.



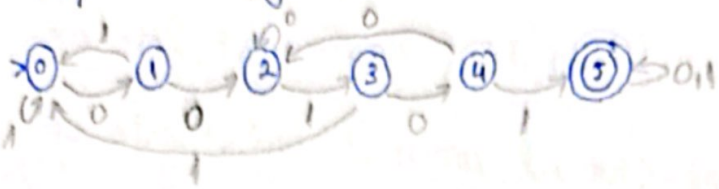

1.4. Limbajul cuvintelor care sunt numere care se divid la 3.



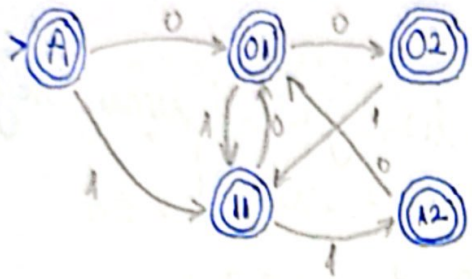



Scanned with CamScanner

1.7 Mulțimea șirurilor binare care au subșirul 00101




2. Descrieți limbajul următorului DFA și simulați evoluția pe intrările $w_1 = 10011110$ și $w_2 = 00110010$.



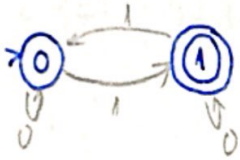

$(A, w_1) \vdash (11, 0011110) \vdash (01, 011110) \vdash (02, 11110) \vdash (11, 1110) \vdash (12, 110)$
stuck

$(A, w_2) \vdash (01, 0110010) \vdash (02, 110010) \vdash (11, 10010) \vdash (12, 0010) \vdash$
$\vdash (01, 010) \vdash (02, 10) \vdash (11, 0) \vdash (12, \varepsilon) \rightarrow$ acceptat.

## Laboratorul 3

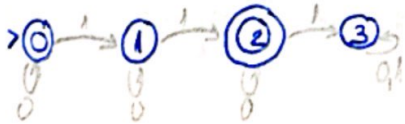
1. Pentru fiecare DFA, scrieți un regex care descrie același limbaj

1.1.



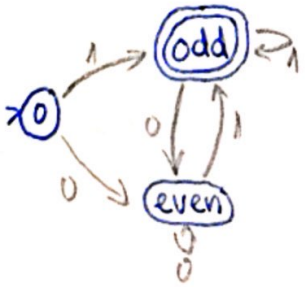

$\rightarrow 0^*10^*(10^*10^*)^*$

1.2.




$\rightarrow 0^*10^*10^*$

1.3.

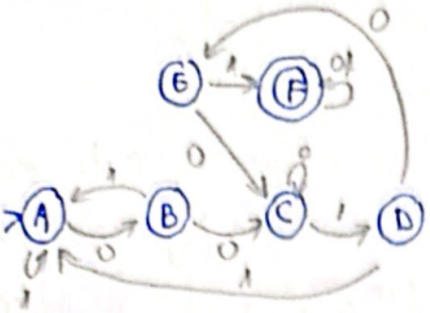



$\rightarrow (0U1)^*1$



Scanned with CamScanner

1.4.




$\rightarrow (0 \cup 1)^* 00 | 01 (0 \cup 1)^*$

2. Calculați:

2.1. $A = \{0^{2k} \mid k \geq 1\}$ ; $B = \{0, \varepsilon\}$ ; $AB = ?$

$A = \{00, 0000, 000000, \dots\}$

$AB = \{00, 000, 0000, 00000, \dots\} = \{0^k \mid k \geq 2\}$

2.2. $A = \{0^n 1^n \mid n \geq 1\}$ ; $B = \{1^n \mid n \geq 1\}$ ; $AB = ?$ ; $BA = ?$

$A = \{01, 0011, 000111, \dots\}$

$B = \{1, 11, 111, 1111, \dots\}$

$AB = \{011, 0111, 01111, \dots, 00111, 001111, \dots\}$

$BA = \{101, 1001, 10001, \dots, 1101, 11001, \dots\}$

2.3. $A = \emptyset$ ; $B = \{1^n \mid n \geq 1\}$ ; $AB = ?$ ; $A^* = ?$ ; $B^* = ?$

$AB = \emptyset$

$A^* = \{\varepsilon\}$

$B^* = \{\varepsilon\} \cup \{1^n\} \cup \{1^{2n}\} \cup \{1^{3n}\} \cup \dots$

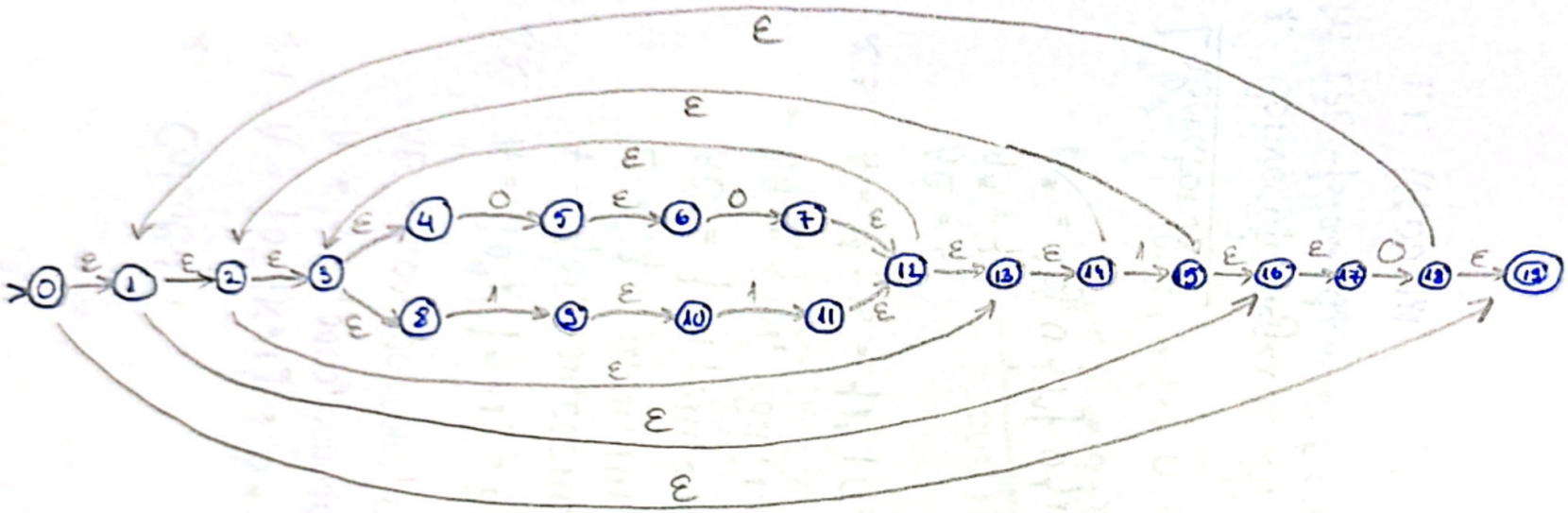
## Laboratorul 5

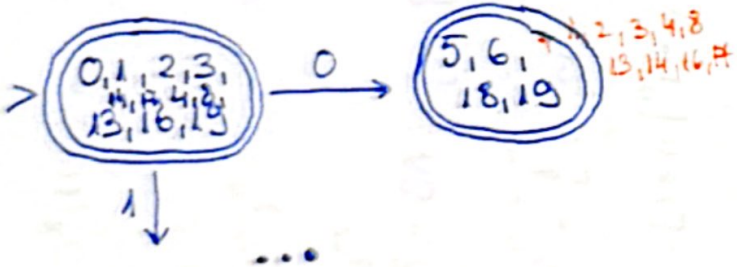
1. Convertiți regexul în NFA folosind Thompson și NFA-ul în DFA folosind subset construction.

1.1. $(((00 \cup 11)^* 1)^* 0)^*$



Scanned with CamScanner




e prea mare nu mai am chef.



Scanned with CamScanner

# Probleme din examene

1. Care dim următoarele cuvinte pot fi folosite, alături de
M $k=0$, pentru a arăta folosind Lema de Pompare că limbajul $\{0^n 1^m 1^m 0^m \mid m, n \geq 0\}$ nu este regulat?

- ☑ $w_n = 0^n 1^m$
- ☑ $w_n = 1^n 0^n$
- ☐ $w_n = 0^n 1^n 0^n 1^n$
- ☐ $w_n = 0^n 1^m 1^m 0^m$
- ☐ Nu există un astfel de cuvânt

• $w_n = 0^n 1^m = \underbrace{000000\ldots0000}_{x\ y}\underbrace{111111\ldots111}_{z}$

$x = 0^a$; $y = 0^b$; $z = 1^m$; $a+b=n$; $b \neq 0$

$xy^k z = 0^{a} 1^m \notin L$ (nr de 0 diferit de nr de 1) $\Rightarrow$

$\Rightarrow$ limbajul nu e regulat

• $w_n = 1^n 0^n = \underbrace{111111111\ldots1111}_{x\ y}\underbrace{000000\ldots000}_{z}$

$x = 1^a$; $y = 1^b$; $z = 0^n$; $a+b=n$; $b \neq 0$

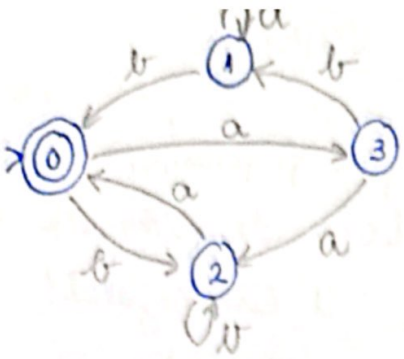
$xy^k z = 1^a 0^n \notin L$ (nr de 0 diferit de nr de 1) $\Rightarrow$

$\Rightarrow$ limbajul nu e regulat.

2. Care din următoarele afirmații sunt adevărate
U privitor la automatul A din imagine:



Scanned with CamScanner




- ☐ Automatul acceptă prin stivă vidă. → nu este PDA
- ☐ Automatul acceptă limbajul vid. → nu
- ☐ Automatul acceptă limbajul $\{a,b\}^*$ → $a \in \{a,b\}^*$ și nu e acceptat
- ☐ Automatul acceptă $\{a^n b^n \mid n \geq 0\}$ → $ab \in \{a^n b^n \mid n \geq 0\}$ și nu e acceptat
- ☐ Automatul acceptă cuvântul aaaaabbbbb → nu
- ☑ Automatul acceptă cuvântul abababab.

$(0, abababab) \vdash_A (3, bababab) \vdash_A (1, ababab) \vdash_A (1, babab) \vdash_A$
$\vdash_A (0, abab) \vdash_A (3, bab) \vdash_A (1, ab) \vdash_A (1, b) \vdash_A (0, \varepsilon) \Rightarrow$ acceptat

3. $L_1$ intersectat cu $L_2 = L_3$. Care afirmații sunt adevărate?

- ☐ Dacă $L_1$ este regulat, atunci $L_3$ este regulat.
- ☐ Dacă $L_1$ și $L_3$ sunt regulate, atunci $L_2$ este independent de context
- ☐ Dacă $L_1$ este independent de context și $L_2$ este independent de context, atunci $L_3$ este independent de context
- ☑ Dacă $L_1$ este regulat și $L_2$ este independent de context, atunci $L_3$ este independent de context.



Scanned with CamScanner

4. Care afirmații sunt adevărate despre gramatica:

U $S \leftarrow aaS \mid Sbb \mid aS \mid Sb \mid \varepsilon$ ?

- ☐ este regulată → nu
- ☐ este în Forma Normală Chomsky → nu
- ☐ generează limbajul $\{a^n b^n \mid n \geq 0\}$
- ☐ generează limbajul $\{a^{2n} b^{2n} \mid n \geq 0\}$
- ☑ generează $L(a^* b^*)$
- ☐ generează $L((aa)^*(bb)^*)$

$S \Rightarrow_G aaS \Rightarrow_G aaSb \Rightarrow_G aab \notin \{a^n b^n \mid n \geq 0\}$
$\notin \{a^{2n} b^{2n} \mid n \geq 0\}$
$\notin L((aa)^*(bb)^*)$

5. Fie următoarea specificație a unui lexer:

M $X : 0^*$

$Y : 1^*$

$Z : 0^* 1^*$

$W : (01)^*$

Care din afirmațiile de mai jos sunt adevărate?

- ☐ analiza lexicală a șirului 0110 va produce tokenii XYYX
- ☐ analiza lexicală a șirului 0110 va produce tokenii ZYX
- ☐ analiza lexicală a șirului 0110 va produce tokenii ZW
- ☐ analiza lexicală a șirului 0101 va produce tokenii ZZ
- ☑ niciuna din cele de mai sus. → și 0110 și 0101 produc 'W'



Scanned with CamScanner

6. Care din următoarele limbaje nu sunt independente de context?

M

- ☐ limbajul cuvintelor binare în care nr de 0 este egal cu nr de 1
- ☑ limbajul peste alfabetul $\{A, C, G, T\}$ în care nr de simboli A, G, C, T apar de același nr de ori
- ☐ limbajul $\{0^n 1^n \mid n \geq 0\}$
- ☐ limbajul peste alfabetul $\{A, C, G, T\}$ în care fiecare simbol apare cel puțin o dată
- ☑ limbajul $\{0^n 1^n 0^n \mid n \geq 0\}$

- $\{0^n 1^n 0^n \mid n \geq 0\}$ nu poate fi implementat doar cu o stivă, deci nu se poate face un PDA.
- $\{A^n C^n G^n T^n \mid n \geq 0\}$ aceeași poveste.

7. Fie A un automat finit determinist. Care afirmații sunt adevărate?

U

- ☐ Dacă A nu conține sink states, atunci acceptă orice cuvânt
- ☑ Dacă A conține un sink state, atunci nu acceptă o infinitate de cuvinte
- ☐ Mulțimea $\{L \mid L$ este acceptat de un DFA cu o singură stare$\}$ conține un element.
- ☑ L(A) este un limbaj regulat
- ☐ Mulțimea $\{L \mid L$ este acceptat de un DFA peste alfabetul $\{0\}\}$ conține un singur element.

Scanned with CamScanner

8. Care dintre următoarele expresii regulate acceptă limbajul generat de gramatica:
U $S \leftarrow aSa \mid aSb \mid bSa \mid bSb \mid \varepsilon$

- [x] $(aa \cup ab \cup ba \cup bb)^*$
- [ ] $(ab \cup ba)^*(aa \cup bb)^*$
- [ ] $(aa)^*(bb)^*(ab)^*(ba)^*$
- [ ] $(ba)^*(ab)^*(aa)^*(bb)^*$
- [ ] $(a \cup b)^*$

- $S \Rightarrow_G aSa \Rightarrow_G aaSba \Rightarrow_G aaba \notin (ab \cup ba)^*(aa \cup bb)^*$
- $S \Rightarrow_G bSa \Rightarrow_G bbSaa \Rightarrow_G bbaa \notin (aa)^*(bb)^*(ab)^*(ba)^*$
- $S \Rightarrow_G aSa \Rightarrow_G abSba \Rightarrow_G abba \notin (ba)^*(ab)^*(aa)^*(bb)^*$
- $a \in (a \cup b)^*$, dar nu poate fi generat de gramatică

9. $L_1$ intersectat cu $L_2$ este un limbaj regulat. Care afirmații
D sunt adevărate?

- [ ] $L_1$ și $L_2$ sunt în mod necesar regulate
- [ ] cel puțin unuia dintre $L_1$ și $L_2$ este în mod necesar independent de context
- [ ] cel puțin unul dintre $L_1$ și $L_2$ este în mod necesar regulat
- [ ] unul dintre $L_1$ și $L_2$ este în mod necesar regulat, iar celălalt independent de context
- [x] $L_1$ poate fi neregulat iar $L_2$ poate fi neregulat.

ex: $L_1 = \{a^n b^n \mid n \geq 0\}$ → neregulat ; $L_2 = \{b^n c^n \mid n \geq 0\}$
$L_1 \cap L_2 = \{a^n \mid n \geq 0\}$ → regulat



Scanned with CamScanner

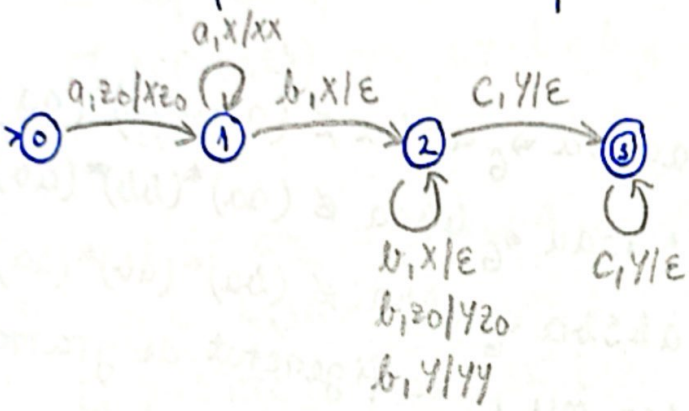
10. Limbajul $\{a^n b^k b^n a^k \mid n, k \geq 0\}$ este

D

- ☐ regulat
- ☑ independent de context
- ☑ dependent de context
- ☑ Turing - acceptat

11. Care afirmații sunt adevărate privitor la PDA-ul de mai jos?

U




- ☐ PDA-ul acceptă cuvântul abc.
- ☐ PDA-ul acceptă cuvântul aaabbbccc
- ☑ PDA-ul acceptă cuvântul aabbbc
- ☐ PDA-ul acceptă cuvinte de forma $a^n b^n$, cu $n > 0$
- ☐ PDA-ul acceptă cuvinte de forma $a^n b^n$, cu $n \geq 0$
- ☐ PDA-ul acceptă cuvinte de forma $a^n b^n c^n$, cu $n > 0$.

$(0, aabbbbc, Z_0) \vdash (1, abbbbc, XZ_0) \vdash (1, bbbbc, XXZ_0) \vdash$

$\vdash (2, bbbc, XZ_0) \vdash (2, bc, Z_0) \vdash (2, c, YZ_0) \vdash (3, \varepsilon, Z_0)$

$\Rightarrow$ PDA-ul acceptă aabbbbc și cuv de forma

$a^n b^{n+m} c^m$, cu $m, n \geq 0$



Scanned with CamScanner

12. Definim următoarea operație peste limbaje cu
D alfabetul $\{0,1\}$: $rep(L) = \{w\#(w) \mid w \text{ in } L\}$, unde $\#(w)$ reprezintă lungimea cuvântului w codificată în binar. Care dintre următoarele afirmații sunt adevărate?

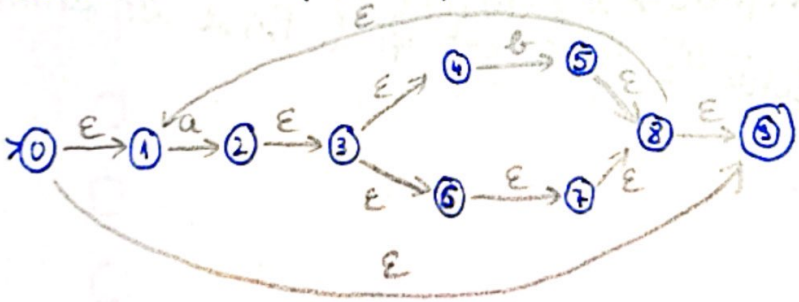
- ☐ dacă L este regulat, atunci rep(L) e regulat
- ☐ rep este o proprietate de închidere peste limbajele regulate
- ☐ $rep(L(0^*))$ este un limbaj regulat
- ☑ $rep(\{\varepsilon\})$ este un limbaj independent de context.

13. Câte stări are automatul mimimal care acceptă
M limbajul $L((ab?)^*)$?

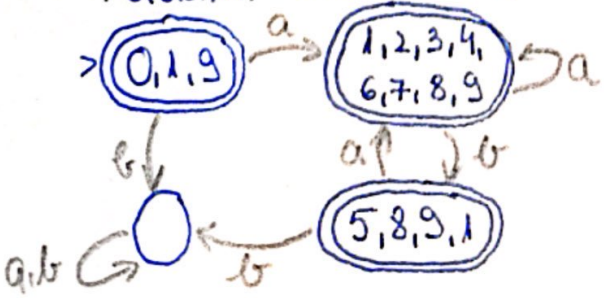
- ☐ 0 stare
- ☐ 2 stări
- ☑ 3 stări
- ☐ 4 stări
- ☐ 5 stări
- ☐ 6 stări
- ☐ 7 stări
- ☐ 8 stări

$(ab?)^* = (a(b \cup \varepsilon))^*$

Folosim Thompson și transformăm în NFA.



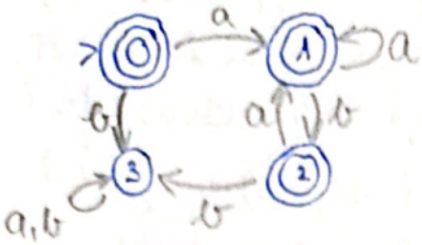

Folosim Subset Construction și transformăm în DFA.



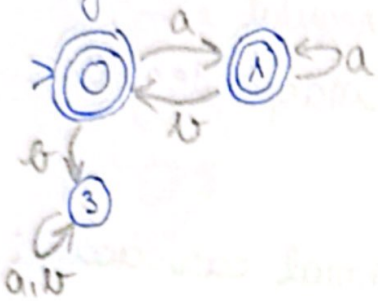



Scanned with CamScanner

Redenumim stările:




Stările 0 și 2 pot fi confundate, deci devin una singură




⇒ automatul minimal are 3 stări.

14. Care dintre următoarele afirmații sunt adevărate?

U
- ☐ mulțimea limbajelor regulate este finită
- ☑ mulțimea limbajelor regulate este infinită, dar numărabilă
- ☐ mulțimea limbajelor independente de context este finită
- ☑ mulțimea limbajelor independente de context este infinită, dar numărabilă.

15. Câte perechi de stări pot fi distinse în următorul DFA?

U



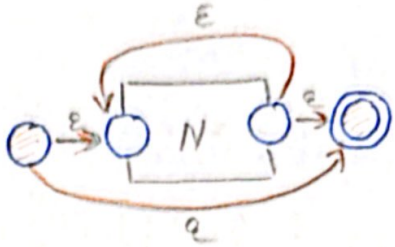

- ☐ 0
- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☑ 5 $(0,3)$; $(1,3)$; $(2,3)$; $(0,1)$; $(0,2)$
- ☐ 6



Scanned with CamScanner

16. Fie S nr de stări și T nr de tranziții noi introduse
U pentru a crea un NFA pentru expresia $e^*$, plecând de la un NFA pentru expresia e. Atunci S+T =

- ☐ 0
- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5
- ☑ 6
- ☐ 7
- ☐ 8
- ☐ 9
- ☐ 10
- ☐ 11
- ☐ 12
- ☐ 13
- ☐ 14




17. Fie următoarea gramatică G:
M $S \leftarrow aSb \mid bSa \mid a \mid b$
Care din următoarele, alături de $A \leftarrow a$, $B \leftarrow b$, sunt gramatici în Forma Normală Chomsky echivalente cu G?

- ☐ $S \leftarrow BX \mid AY$ ; $X \leftarrow SA$ ; $Y \leftarrow BS$
- ☐ $S \leftarrow BY \mid AX \mid a \mid b$ ; $X \leftarrow AS$ ; $Y \leftarrow BS$
- ☑ $S \leftarrow AX \mid BY \mid a \mid b$ ; $X \leftarrow SB$ ; $Y \leftarrow SA$
- ☐ $S \leftarrow AX \mid AY \mid a \mid b$ ; $X \leftarrow BS$ ; $Y \leftarrow SB$
- ☐ $S \leftarrow AX$ ; $X \leftarrow BY$ ; $Y \leftarrow a \mid b \mid S$
- ☐ niciuna din cele de mai sus



Scanned with CamScanner

18. Care dintre următoarele limbaje sunt regulate?

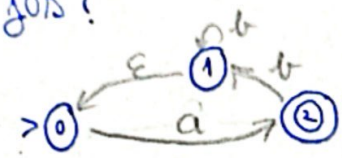
U

- ☐ $\{a^n b^n \mid n \geq 1500\}$
- ☑ $\{a^n b^n \mid n \leq 1500\}$
- ☑ $\{a^n b^m \mid m+m = 1500\}$
- ☐ $L(a^*)\{a^n b^n \mid n \geq 0\}$
- ☐ $\{a^n b^n \mid n \geq 0\}\ L(b^*)$
- ☑ $\{a^n \mid n \geq 0\} \cup \{b^n \mid n \geq 0\}$

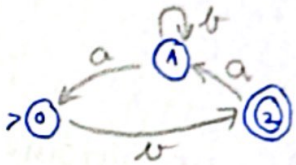
19. Care afirmații sunt adevărate relativ la automatele de mai jos?

M

$A_1$:




$A_2$:




- ☐ $L(A_1) = L(A_2)$
- ☐ $L(A_1) = L(ab^*a)$
- ☐ $L(A_1) = L(ba^*b)$
- ☐ $L(A_2) = L(bab^*ab)$
- ☑ $L(A_2) = L(b(ab^*ab)^*)$
- ☐ $L(A_1)$ inclus în $L(A_2)$
- ☐ $L(A_2)$ inclus în $L(A_1)$



Scanned with CamScanner